

SHOPNEAR

MINOR PROJECT REPORT

A Hyperlocal Shop-Discovery and Reservation Platform

Built on an Availability-Confidence Model

Submitted in partial fulfilment of the requirement for the Degree of
Bachelor of Engineering in Computer Science & Engineering



PARUL UNIVERSITY, VADODARA, GUJARAT (INDIA)

Submitted By:

Vagisha (2303051050934)

Harshad Ghadge (2303051051432)

Saksham (2303051050813)

Aman (2303051050981)

Under the Guidance of:

Mr. Ravi Ranjan

(Assistant Professor, CSE)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PARUL INSTITUTE OF TECHNOLOGY

VADODARA, GUJARAT

SESSION: AY 2026–2027

Parul University
Parul Institute of Technology

(Session: 2026–2027)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that **Vagisha (2303051050934)**, **Harshad Ghadge (2303051051432)**, **Saksham (2303051050813)** and **Aman (2303051050981)**, students of **CSE VI Semester** of Parul Institute of Technology, Vadodara, have completed the Minor Project titled “**ShopNear — A Hyperlocal Shop-Discovery and Reservation Platform**”, as per the syllabus, and have submitted a satisfactory report on this project as a partial fulfilment towards the award of the degree of **Bachelor of Technology in Computer Science and Engineering** under Parul University, Vadodara, Gujarat (India).

The work presented in this report is a record of original work carried out by the above candidates under our supervision during the academic session 2026–2027.

Mr. Ravi Ranjan
(Project Guide)
(Assistant Professor, CSE)

Asst. Prof. Sumitra Menaria
Head (CSE)
PIT, Vadodara

DECLARATION

We, the undersigned, solemnly declare that the project report titled “*ShopNear — A Hyperlocal Shop-Discovery and Reservation Platform*” is based on our own work carried out during the course of our study, under the supervision of **Mr. Ravi Ranjan**, Assistant Professor, Department of Computer Science & Engineering.

We affirm that the statements made and the conclusions drawn are the outcomes of our own work. We further certify that:

- The work contained in this report is original and has been carried out by us under the general supervision of our supervisor.
- This work has not been submitted to any other institution for the award of any degree, diploma or certificate in this or any other university in India or abroad.
- We have followed all the guidelines provided by the university in writing this report. Wherever we have used material (data, theoretical analysis or text) from other sources, we have given due credit to them in the text of the report and have included their details in the references.
- The system described in this report operates entirely on locally generated demonstration data. No real shop, merchant, customer or monetary transaction is involved at any point, and every screenshot reproduced in Chapter V was captured from the application actually running on a development machine.

Vagisha [2303051050934]

SIGNATURE _____

Harshad Ghadge [2303051051432]

SIGNATURE _____

Saksham [2303051050813]

SIGNATURE _____

Aman [2303051050981]

SIGNATURE _____

ACKNOWLEDGEMENT

In this semester we have completed our project on **ShopNear**, a hyperlocal shop-discovery and reservation platform for neighbourhood retail. During this period all the group members worked collaboratively on the project and learnt how software of this kind is designed, built and defended: how a relational schema is shaped around a genuine product decision, how geospatial queries are indexed so that they stay fast, how a state machine keeps a transactional workflow honest, and how an automated test suite turns a claim into evidence.

We gratefully acknowledge the assistance, cooperation, guidance and clarification provided by **Mr. Ravi Ranjan** during the development of this project. We would also like to thank our Head of Department, **Prof. Sumitra Menaria**, and our Principal, **Dr. Swapnil Parikh**, for giving us the opportunity to develop this project. Their continuous motivation and guidance helped us overcome the various obstacles we met while completing it.

We perceive this project as an opportunity and a significant milestone in our career development. We will strive to use the skills and knowledge gained here in the best possible way, and we will continue to work to improve them.

Vagisha [2303051050934]

SIGNATURE _____

Harshad Ghadge [2303051051432]

SIGNATURE _____

Saksham [2303051050813]

SIGNATURE _____

Aman [2303051050981]

SIGNATURE _____

TABLE OF CONTENTS

CHAPTER	TOPIC	PAGE
	ABSTRACT	11
Chapter I	INTRODUCTION	13
1.1	Overview	13
1.2	Problem Statement	13
1.3	Objectives of the Project	14
1.4	Applications and Scope	15
1.5	Organization of the Report	16
Chapter II	LITERATURE SURVEY	17
2.1	Introduction	17
2.2	Existing Systems	17
2.3	Review of Enabling Technologies	18
2.4	Comparative Study	19
2.5	Research Gap Identified	20
Chapter III	METHODOLOGY	21
3.1	Overview of Methodology	21
3.2	Platforms and Technologies Used	21
3.3	Proposed Methodology	23
3.4	Project Modules	24
3.5	System Architecture	25
3.6	Use Case Diagram	25
3.7	Data Flow Diagrams	26
3.8	Entity-Relationship Diagram	28
3.9	The Availability Confidence Model	29
3.10	Geospatial Search and Composite Ranking	31
3.11	Multi-Item Search Algorithm	32
3.12	Order State Machine	32
Chapter IV	SYSTEM REQUIREMENTS	34
4.1	Software Requirements	34
4.2	Hardware Requirements	35
4.3	Functional Requirements	36
4.4	Non-Functional Requirements	37
Chapter V	IMPLEMENTATION AND RESULTS	39
5.1	Introduction	39
5.2	Database Layer	39
5.3	Backend API Development	40
5.4	Customer Application	42
5.5	Merchant Application	45

CHAPTER	TOPIC	PAGE
5.6	Admin Console	48
5.7	Realtime Layer and Scheduled Jobs	51
5.8	Security Implementation	52
5.9	Deployment Topology	53
Chapter VI	TESTING AND VALIDATION	55
6.1	Testing Strategy	55
6.2	Distribution of the Test Suite	55
6.3	Representative Test Cases	56
6.4	Test Execution Results	58
6.5	End-to-End Journey Verification	58
Chapter VII	CONCLUSION AND FUTURE SCOPE	60
7.1	Conclusion	60
7.2	Key Findings	60
7.3	Limitations	61
7.4	Future Scope	62
Chapter VIII	REFERENCES	64

LIST OF FIGURES

S. No	Figure	Title	Page
1	Fig. 3.1	System architecture of ShopNear	25
2	Fig. 3.2	Use case diagram	26
3	Fig. 3.3	Data flow diagram — Level 0 (context diagram)	27
4	Fig. 3.4	Data flow diagram — Level 1	28
5	Fig. 3.5	Entity-relationship diagram	29
6	Fig. 3.6	Availability confidence decay over time	30
7	Fig. 3.7	Order state machine	33
8	Fig. 5.1	Customer app — location gate	43
9	Fig. 5.2	Customer app — neighbourhood home feed	43
10	Fig. 5.3	Customer app — search results with confidence badges	44
11	Fig. 5.4	Customer app — multi-item shopping-list search	44
12	Fig. 5.5	Customer app — cart and order history	45
13	Fig. 5.6	Customer app — responsive desktop layout	45
14	Fig. 5.7	Merchant app — login and registration wizard	46
15	Fig. 5.8	Merchant app — daily summary dashboard	47
16	Fig. 5.9	Merchant app — bulk stock management with barcode entry	47
17	Fig. 5.10	Merchant app — interface rendered in Gujarati	48
18	Fig. 5.11	Admin console — login	49
19	Fig. 5.12	Admin console — analytics dashboard	50
20	Fig. 5.13	Admin console — ranking weight and decay threshold controls	51
21	Fig. 5.14	Admin console — dispute resolution queue	51
22	Fig. 5.15	Deployment topology: three bundles, one origin	54
23	Fig. 6.1	Test suite execution output	58
24	Fig. 7.1	Honest gap labelling in the admin console	62

LIST OF TABLES

S. No	Table	Title	Page
1	Table 2.1	Comparison of existing retail and quick-commerce platforms	20
2	Table 3.1	Technology stack and the role of each component	23
3	Table 3.2	Availability badge derivation rules	29
4	Table 3.3	Default composite ranking weights	31
5	Table 3.4	Legal order state transitions	33
6	Table 4.1	Software requirements	35
7	Table 4.2	Hardware requirements	36
8	Table 4.3	Non-functional requirements and how each is met	38
9	Table 5.1	Seeded demonstration dataset	40
10	Table 5.2	REST API endpoints implemented	42
11	Table 5.3	Source code size by module	53
12	Table 6.1	Distribution of automated tests	56
13	Table 6.2	Representative test cases and outcomes	58
14	Table 6.3	Recorded end-to-end reservation journey	59

LIST OF ABBREVIATIONS

S. No	Abbreviation	Full Form
1	API	Application Programming Interface
2	CORS	Cross-Origin Resource Sharing
3	CRUD	Create, Read, Update, Delete
4	CSS	Cascading Style Sheets
5	DBMS	Database Management System
6	DFD	Data Flow Diagram
7	ER	Entity-Relationship
8	GiST	Generalised Search Tree (PostgreSQL index type)
9	GIN	Generalised Inverted Index (PostgreSQL index type)
10	GPS	Global Positioning System
11	GUI	Graphical User Interface
12	HTML	Hyper Text Markup Language
13	HTTP	Hyper Text Transfer Protocol
14	i18n	Internationalisation
15	JS	JavaScript
16	JSON	JavaScript Object Notation
17	JWT	JSON Web Token
18	ONDC	Open Network for Digital Commerce
19	ORM	Object-Relational Mapping
20	OTP	One-Time Password
21	POS	Point of Sale
22	PWA	Progressive Web Application
23	REST	Representational State Transfer
24	SQL	Structured Query Language
25	SRID	Spatial Reference System Identifier
26	TS	TypeScript
27	UI / UX	User Interface / User Experience

S. No	Abbreviation	Full Form
28	WCAG	Web Content Accessibility Guidelines

ABSTRACT

A customer who needs a packet of *atta*, a litre of milk or a phone charger today has two poor options. They can walk to the nearest neighbourhood shop and hope the item is on the shelf, or they can open a quick-commerce application that will deliver the item from a warehouse that is not a neighbourhood shop at all. The first option wastes trips; the second bypasses the very shops that make a neighbourhood work. **ShopNear** addresses the gap between the two: it lets a customer search for an everyday item and see which nearby shops actually carry it, at what price, and — critically — how recently that availability was confirmed.

The central design decision of this project is that **ShopNear never promises a live stock count**. A small shop cannot maintain one; asking it to do so guarantees the data will be wrong. Instead, availability is modelled as a *confidence level paired with a timestamp*. The shopkeeper answers a single question — “Have it” or “Don’t have it” — with one tap, and the customer sees a badge derived from that answer and its age: *In stock — confirmed 20 minutes ago*, *Likely available*, *Usually available*, or *Ask the shop*. A scheduled job decays stale signals automatically, so confidence degrades honestly rather than silently going stale. No numeric stock count is displayed anywhere in the customer interface.

The system is implemented as a TypeScript monorepo. A single Node.js and Express REST API, backed by PostgreSQL 16 with the PostGIS and `pg_trgm` extensions and accessed through the Prisma ORM, serves three separate React clients: a customer storefront, a merchant console and an administrator panel. PostGIS provides true geodesic radius search over a GiST-indexed geography column, while `pg_trgm` supplies typo tolerance so that “ata”, “atta” and “aata” all resolve to wheat flour. Search results are ordered by a composite score that combines availability confidence, proximity, shop rating and whether the shop is open, with the weights adjustable at runtime. A distinguishing feature is *multi-item search*: given a shopping list, the system identifies the single nearby shop that satisfies the largest number of items and proposes a two-shop split for the remainder.

Transactions follow a *reserve-and-collect* model. Every status change passes through an explicit state machine that rejects illegal transitions, orders expire automatically if unanswered, and each availability change is written to an append-only audit log. Security measures include Argon2 password hashing, JWT access and refresh tokens, role-based middleware, schema validation on every endpoint and rate limiting on authentication routes.

The delivered system comprises approximately 29,000 lines of TypeScript across the API, three clients and a shared contract package, and is validated by an automated suite of **379 tests across 35 test files, all of which pass**. Coverage includes every legal and illegal order state transition, cross-tenant authorisation checks, the confidence derivation rules and the deterministic seed generator.

The entire platform runs offline on a single laptop after one setup command, using a deterministic demonstration dataset of 342 products, 14 shops, 1,749 inventory records and 120 historical orders.

CHAPTER I

INTRODUCTION

1.1 Overview

ShopNear is a hyperlocal shop-discovery and reservation platform for neighbourhood retail. It answers one narrow, everyday question with precision: *which shop near me has this item right now, at what price, and how much should I trust that answer?*

The neighbourhood shop — the kirana, the general store, the stationer, the chemist — remains the backbone of Indian retail. It is close, it extends informal credit, it knows its customers by name, and it stocks exactly what the surrounding streets actually buy. What it does not have is any digital presence worth the name. A customer standing two hundred metres away has no way to know whether the shop has the item they need. The result is a walk that may be wasted, and over time a slow drift of that customer towards a warehouse-backed delivery application.

The obvious technical response — give every shop an inventory system with live stock counts — is the wrong one, and recognising why is the intellectual core of this project. A shop with a single counter, a queue of customers and no barcode scanner cannot maintain a live count. If the system demands one, the count will be wrong within a day, and a wrong count is worse than no count at all: it destroys the customer's trust in the platform and the shopkeeper's willingness to use it.

ShopNear therefore makes a different promise. It does not promise an accurate stock *number*; it promises a *fast confirmation*. Availability is stored as a confidence level with a timestamp, and the interface always displays the age of that information alongside it. A shopkeeper confirms or declines an item with a single tap, and that tap is what updates availability. Where no recent signal exists, the system says so plainly — *Ask the shop* — and still allows the customer to place a reservation. Honesty about uncertainty is treated as a feature, not a shortcoming.

The platform is built as three separate web clients over one REST API. Customers search and reserve; merchants confirm orders and manage their catalogue in English, Hindi or Gujarati; administrators moderate shops, resolve disputes and tune the search ranking. The primary transaction is *reserve and collect*: the shop sets the item aside, the customer walks over and pays in person. Delivery exists as a secondary, opt-in flow fulfilled by the shop's own delivery person. There is no platform-owned warehouse, no delivery fleet and no commission model, because none of those things serve the problem being solved.

1.2 Problem Statement

Neighbourhood retail in India is digitally invisible in a way that its physical prominence makes surprising. The specific failures are as follows.

The customer cannot see inside the shop. Proximity is worthless without information. A shop fifty metres away that stocks the required item is functionally equivalent to a shop that does not, because the customer has no way to distinguish them without walking there. This converts every small purchase into a gamble and pushes customers towards platforms that at least tell them what is in stock.

Existing platforms do not solve this for the shop. Large marketplaces such as Amazon and Flipkart are national in scale and brand-driven in ranking; a single neighbourhood kirana is invisible within them. Quick-commerce applications such as Blinkit, Zepto and Swiggy Instamart do not list neighbourhood shops at all — they operate their own dark stores, and the local shop is a competitor rather than a participant. Both charge commissions that a business running on grocery margins cannot absorb, and both require onboarding procedures that assume a level of digital literacy and documentation many shopkeepers do not have.

Inventory systems ask the wrong thing of the shopkeeper. Conventional e-commerce and point-of-sale software is built around an authoritative stock quantity. Maintaining that number requires every sale to be recorded at the moment it happens. In a shop where transactions are cash, fast and unrecorded, this is not a training problem to be solved — it is a structural mismatch. Any design that depends on it will produce data that is confidently wrong.

Stale data is presented as fresh. Systems that do capture availability almost universally display it without any indication of age. A record last touched three weeks ago looks identical to one confirmed five minutes ago. The customer is given no basis on which to calibrate their trust, so the first wrong answer discredits every subsequent right one.

Language and literacy are barriers on the merchant side. Merchant-facing tooling is overwhelmingly English-only and assumes comfort with dense forms and tabular data. For a shopkeeper more fluent in Gujarati or Hindi than in English, this alone is enough to prevent adoption regardless of the product's merits.

What is required is a platform that asks the shopkeeper for the least possible effort — one tap — while giving the customer a signal whose reliability is explicitly stated rather than implied. That is the problem ShopNear sets out to solve.

1.3 Objectives of the Project

1. **To design and implement an availability confidence model** in which stock information is represented as a state paired with a timestamp, and the label presented to the customer is derived from both, so that the age of the information is always visible rather than hidden.
2. **To build a geospatial discovery engine** that returns shops and products within a user-selected radius using true geodesic distance over a spatially indexed database, rather than approximate bounding-box filtering.

3. **To rank results by a composite score** combining availability confidence, proximity, shop rating and opening status, with the weights held in runtime configuration so that ranking behaviour can be examined and adjusted without redeployment.
4. **To provide typo-tolerant and transliteration-aware search**, so that a query typed as “ata”, “atta”, “aata” or “ઁલ્લ” resolves to the same product.
5. **To implement multi-item shopping-list search** that identifies the single nearby shop covering the greatest number of requested items and proposes a two-shop split for the remainder — a capability that warehouse-based quick-commerce cannot offer, since it has only one fulfilment point.
6. **To enforce order integrity through an explicit state machine** in which every status change is validated against a declared transition table, illegal transitions are rejected at the domain layer, and unanswered reservations expire automatically.
7. **To deliver a merchant experience optimised for speed and low English literacy**, including full interface translation into Hindi and Gujarati, large single-tap confirmation controls, and bulk catalogue management with barcode entry.
8. **To maintain a complete audit trail** of availability changes and search activity, supporting both the customer-facing confidence badges and administrative analytics such as unmet-demand reporting.
9. **To validate the system with an automated test suite** covering the domain logic, every legal and illegal order transition, and cross-tenant authorisation boundaries, so that correctness claims are supported by reproducible evidence.

1.4 Applications and Scope

Neighbourhood retail of every kind. The data model supports nine shop categories — kirana, general store, stationery, hardware, chemist, bakery, dairy, farsan and vegetable — each with its own curated starter catalogue so that a newly approved shop is never empty on its first day. The design generalises to any small retailer whose stock is predictable in kind but variable in the moment.

For customers, the platform converts a guess into information. Beyond single-item lookup, the shopping-list feature answers a question that quick-commerce structurally cannot: *which one shop can I visit to get most of what I need?* Because ShopNear indexes many independent shops rather than one warehouse, minimising the number of stops is a meaningful and solvable optimisation.

For merchants, the platform offers digital visibility at effectively zero operational cost. There is no commission, no packaging requirement and no delivery obligation. The daily workload is a small number of one-tap confirmations. The merchant console additionally surfaces demand the shop is failing to capture — the item customers most often request and are most often refused — turning the platform into a source of stocking intelligence rather than merely a sales channel.

For the local economy, the model keeps both the transaction and the margin within the neighbourhood. The customer pays the shop directly and in person; no intermediary takes a cut, and no warehouse displaces the shop.

Boundaries of the present work. This is a demonstration system operating on generated data. It uses no paid third-party services, integrates no live payment gateway and performs no verification of merchant documents. The full scope and its limits are stated in Section 7.3, and the path to a production system in Section 7.4.

1.5 Organization of the Report

Chapter I introduces the project, states the problem, lists the objectives and defines the scope.

Chapter II surveys existing marketplace, quick-commerce and local-directory systems, reviews the enabling technologies, compares the alternatives feature by feature and identifies the gap this project addresses.

Chapter III presents the methodology: the system architecture, the technology choices and their justification, the functional modules, the design diagrams (use case, data flow, entity-relationship and state machine), and detailed treatment of the three algorithmic contributions — the confidence model, composite ranking and multi-item search.

Chapter IV specifies the software and hardware requirements and enumerates the functional and non-functional requirements against which the system is judged.

Chapter V documents the implementation and results, layer by layer, illustrated with screenshots captured from the running application.

Chapter VI describes the testing strategy, the composition of the automated test suite and the execution results.

Chapter VII concludes, states the limitations honestly and sets out the future scope.

Chapter VIII lists the references.

CHAPTER II

LITERATURE SURVEY

2.1 Introduction

Digital commerce in India has expanded rapidly, but the expansion has been uneven. Investment and engineering effort have concentrated on two models: the national marketplace, which aggregates sellers at scale, and quick commerce, which compresses delivery time by placing inventory in dense networks of dark stores. Both are demonstrably successful. Neither serves the neighbourhood shop, and neither answers the question a customer asks when they are already within walking distance of one.

This chapter reviews the systems that occupy the space around ShopNear, examines the technologies that make an alternative feasible, compares the options against criteria that matter to a small retailer, and states the gap that remains.

2.2 Existing Systems

2.2.1 *National marketplaces*

Amazon and Flipkart operate catalogue-centric marketplaces at national scale, using centralised warehousing, algorithmic recommendation and integrated logistics. Their ranking optimises for conversion across a national inventory; physical proximity is essentially irrelevant to it. A neighbourhood shop can in principle list on these platforms, but it competes on price and rating against sellers with warehouse economics, and the customer relationship is mediated by the platform brand. The commission structure and fulfilment requirements are calibrated for sellers operating at volume.

2.2.2 *Quick commerce*

Blinkit, Zepto and Swiggy Instamart deliver a limited catalogue within minutes by operating their own dark stores — compact, non-public warehouses positioned throughout a city. Availability data in these systems is genuinely accurate because the platform controls the inventory directly. The consequence is that the neighbourhood shop does not appear in the system at all; it is competition, not supply. Reported operating models depend on substantial per-order subsidy and on stocking only fast-moving lines, which is a different proposition from the long tail a kirana actually carries.

2.2.3 *Local directories and discovery services*

JustDial, Google Maps and magicpin list local businesses, providing name, address, opening hours, contact details and reviews. This solves discovery of the *shop*, but not of the *item*. None of these services model per-shop inventory, so the customer still cannot determine whether a particular

product is available without telephoning or visiting. Listing data is also frequently stale, and no age indication is offered.

2.2.4 ONDC

The Open Network for Digital Commerce is a government-backed protocol initiative intended to decouple buyers and sellers from any single platform by standardising commerce interactions across a shared network. Its ambition is directly relevant to the problem addressed here. It is, however, a network protocol rather than an application: it specifies how catalogue, order and fulfilment messages are exchanged, and it presumes participants can publish a structured catalogue with meaningful availability data. The question of how a single-counter kirana produces that data reliably is left to the seller-side application — which is precisely the question this project takes as its subject.

2.2.5 Informal digital commerce

In practice, a large volume of neighbourhood retail already runs over WhatsApp: the customer messages a list, the shopkeeper replies about availability, and payment is made on collection or delivery. This is worth taking seriously, because it reveals the true baseline. It works because it demands nothing of the shopkeeper beyond replying to a message. It fails because it is not searchable, offers no discovery for a customer who does not already know the shop, does not scale past a handful of regulars, and leaves no structured record. ShopNear can be read as an attempt to preserve the low effort of this interaction while adding the discoverability and structure it lacks.

2.3 Review of Enabling Technologies

PostGIS and geodesic search. PostGIS extends PostgreSQL with spatial types and operators. Storing shop locations as `geography(Point, 4326)` allows `ST_DWithin` to express “within r metres” as a true distance on the ellipsoid, and a GiST index makes the predicate index-accelerated rather than a sequential scan. The alternative — computing the haversine formula in application code — cannot use an index and degrades linearly with the number of shops.

Trigram matching for typo tolerance. The `pg_trgm` extension decomposes strings into three-character sequences and measures similarity by trigram overlap, supported by a GIN index. This is what allows “ata” to match “Aashirvaad Atta 5 kg”. For an application whose users type transliterated Hindi and Gujarati into a Latin-script field, tolerance of spelling variation is not a refinement but a basic requirement.

Stateless authentication. JSON Web Tokens (RFC 7519) carry signed claims, allowing an API to authenticate a request without a server-side session lookup. Paired with a short-lived access token and a longer-lived refresh token, this gives three separate clients a uniform authentication mechanism. Argon2, winner of the Password Hashing Competition, is used for password storage; it is deliberately memory-hard, which raises the cost of offline brute-force attack relative to older algorithms.

Bidirectional realtime transport. Socket.IO provides an event-driven channel over WebSocket with automatic fallback and reconnection. When a merchant confirms an order, the customer’s screen must update without a refresh. Short-interval polling achieves the same visible result at small scale and is simpler to reason about; a push channel becomes the better trade-off as the number of idle open sessions grows. Both are present in this system — see Section 5.7.

Type-safe data access and shared contracts. Prisma generates a fully typed client from a declarative schema, so a query returning a field that does not exist fails at compile time rather than in production. Zod schemas placed in a shared workspace package are used both to validate incoming requests on the server and to type responses on the client, so that a single definition governs both ends of every contract.

2.4 Comparative Study

The table below compares the available approaches against criteria that determine whether a small retailer can realistically participate, and whether a customer receives trustworthy information.

System	Local shop visibility	Per-shop item availability	Freshness shown	Merchant effort	Commission
National marketplace (Amazon, Flipkart)	Low — ranking is brand and conversion driven; proximity is not a factor	Yes, but warehouse-level, not shop-level	No age indication	High — listings, packaging, fulfilment SLAs	High
Quick commerce (Blinkit, Zepto, Instamart)	None — dark stores replace local shops	Accurate, but for platform-owned stock only	Not applicable	Not applicable — shops do not participate	Not applicable
Local directory (JustDial, Google Maps)	High for the shop as a business	None — no inventory model	No; listing data often stale	Low — a one-time listing	Advertising fees
Traditional offline retail	Physical footfall only	Manual — the customer must ask in person	Perfectly fresh, but only on arrival	None	None
WhatsApp ordering	Existing customers only	Conversational, unstructured	Implicit in the conversation	Very low	None

System	Local shop visibility	Per-shop item availability	Freshness shown	Merchant effort	Commission
ShopNear (proposed)	High — ranked by proximity and confidence, direct shop-to-customer	Yes — per shop, per product, with explicit confidence state	Yes — age is displayed and decays automatically	Very low — one tap per requested item	None

Table 2.1 — Comparison of existing retail and quick-commerce platforms

2.5 Research Gap Identified

Reading down the “freshness shown” column of Table 2.1 isolates the gap. Every system either has no per-shop availability data at all, or presents whatever data it has as though it were current. No reviewed system treats the *age* of an availability signal as a first-class quantity that is displayed to the user and decays automatically over time.

The second gap is one of effort. Systems that do model inventory require the merchant to maintain an authoritative quantity. The systems that demand nothing of the merchant — directories, WhatsApp — provide no structured availability at all. No reviewed system occupies the middle position: a structured, searchable availability signal obtained at a cost of one tap.

The third gap concerns the shape of the customer’s actual problem. A household does not buy one item; it buys a list. A platform indexing many independent shops can meaningfully minimise the number of stops that list requires, whereas a single-warehouse system has no such problem to solve and therefore no such feature. This optimisation appears in none of the reviewed systems.

ShopNear is positioned precisely in these three gaps: **timestamped confidence rather than asserted stock, one-tap merchant effort rather than inventory maintenance, and list-level optimisation across many shops rather than item-level fulfilment from one.**

CHAPTER III

METHODOLOGY

3.1 Overview of Methodology

ShopNear is built as a **decoupled, three-client monorepo**. A single REST API owns all business logic and all database access; three independent React applications consume it. The three clients are deliberately separate deployments rather than one application with a role switch, because the customer, the shopkeeper and the administrator have almost nothing in common in their goals, their vocabulary or their device. Each client has its own route guard and its own login screen, which removes an entire class of confusion. The separation is architectural rather than a matter of addressing: for deployment the three bundles are served from a single origin at different paths, and the user chooses their role at login (Section 5.9).

Development followed an **incremental, phase-gated process**. Each phase produced a working, demonstrable slice rather than a horizontal layer, and no phase was considered complete until it could be exercised through a user interface and covered by automated tests. The phases were: (1) database foundation and deterministic seed; (2) API core — authentication, search, orders, reviews, disputes; (3) customer application; (4) merchant application; (5) administrator console. The rule applied throughout was that *a feature that exists only in the database is not done*.

Two architectural decisions taken early proved to be load-bearing for everything that followed, and both are worth stating explicitly.

The global catalogue is separated from per-shop inventory. A `Product` record is an idea — “Amul Butter 500 g” — held once, globally, with its brand, unit, barcode and search keywords. A `ShopInventory` record is what makes that idea real in one particular shop: it carries the price, the availability state and the timestamp. Without this separation, every shop would hold its own spelling of every product, and searching for “atta” across fourteen shops would mean matching fourteen unrelated strings. With it, one product resolves to many priced, dated offers and cross-shop comparison becomes a simple join.

No business code reads the system clock directly. All time-dependent logic — badge derivation, reservation expiry, the decay job — obtains the current instant from a single `clock` service backed by a persisted offset. This was introduced in the very first API phase rather than added later as a demonstration convenience, because retrofitting it would have meant auditing every call site. It allows the entire system to be advanced through time on demand, which is what makes the confidence model observable within a short demonstration rather than over two days of real waiting.

3.2 Platforms and Technologies Used

Every choice below was made against three criteria: it must run offline on a student laptop, it must be defensible in a viva, and it must be conventional rather than clever wherever a conventional

option exists.

Layer	Technology	Role and justification
Language	TypeScript 5.6	One language across API, all three clients and the shared contract package; compile-time checking of every API payload shape
Runtime	Node.js 22 LTS	Non-blocking I/O suited to a request-per-query workload
API framework	Express 5	Minimal, explicit middleware pipeline; each concern is a visible, orderable layer
Database	PostgreSQL 16	Relational integrity for orders and inventory; mature extension ecosystem
Spatial extension	PostGIS 3.4	<code>geography(Point, 4326)</code> with GiST index enables true geodesic radius search that uses an index
Text search extension	<code>pg_trgm</code>	Trigram similarity with GIN index for typo-tolerant and transliterated product matching
ORM	Prisma 6	Declarative schema, versioned migrations, generated type-safe client; raw SQL retained where PostGIS requires it
Frontend	React 18 + Vite 5	Component model with fast development builds; three separate client bundles
Routing / server state	React Router 6, TanStack Query 5	Declarative routing; caching, revalidation and request de-duplication
Styling	Tailwind CSS 3	Utility-first styling keeping one visual system across three applications
Authentication	JWT + Argon2	Stateless access and refresh tokens; memory-hard password hashing
Realtime	Socket.IO 4	Authenticated push channel with per-shop and per-order rooms (server-side; see Section 5.7)
Validation	Zod 4	One schema per contract, shared by client and server via the workspace package
Scheduled jobs	node-cron	Hourly availability decay; reservation expiry sweep
Barcode input	@zxing/browser	Camera-based barcode scanning in the merchant console, with text fallback
Charts	Recharts 2	Administrative analytics

Layer	Technology	Role and justification
Internationalisation	i18next	Full English, Hindi and Gujarati coverage in the merchant console
Testing	Vitest 2, Supertest 7	Unit tests for domain logic; HTTP-level integration tests against the real database
Orchestration	Docker Compose	Reproducible PostGIS instance; one command brings up the database

Table 3.1 — Technology stack and the role of each component

3.3 Proposed Methodology

The operational cycle of the system is best understood by following a single reservation from search to collection.

Step 1 — Location is established. The customer application requires a location before it will display any catalogue at all. Discovery is meaningless without an anchor point, so this is a hard gate rather than a preference. The chosen point and search radius are held in client state and sent with every discovery request.

Step 2 — The query is resolved to candidate offers. The API receives the search term, the coordinates and the radius. It first narrows to shops within the radius using `ST_DWithin` against the GiST-indexed geography column, then matches the term against product names and the curated `searchKeywords` array, falling back to trigram similarity so that misspellings and transliterations still resolve. The result is a set of shop-product pairs, each with a price, an availability state and a timestamp.

Step 3 — Confidence is derived and results are ranked. For each candidate, the availability state and the age of its timestamp are passed through the badge rules of Section 3.9 to produce a customer-facing label. The badge is then mapped to a numeric confidence value which is combined with normalised proximity, shop rating and opening status to produce the composite score of Section 3.10. Results are ordered by that score. The numeric score is used internally for ordering only and is never sent to the client.

Step 4 — A reservation is placed. The customer adds items to a cart and checks out against a single shop. The API creates the order inside one database transaction, writing a snapshot of each product name, unit label and unit price onto the order line, so that a historical order never depends on live catalogue data that may since have changed. A four-digit pickup code is generated, and an expiry instant is set two hours ahead.

Step 5 — The merchant responds. The order appears on the merchant console within a few seconds, accompanied by an audible alert. The merchant marks each line *Have it* or *Don't have it* and confirms. This is the system's single most important interaction and is designed for completion in under eight seconds. Each answer does double duty: it resolves the order line, and it updates that

shop's availability record for that product, writing an entry to the append-only `AvailabilityEvent` log. Merchant effort is thus converted directly into data freshness for every future customer.

Step 6 — The customer is updated and collects. The status change reaches the customer's open tracking screen without any refresh. For a reserve-and-collect order, confirmation advances automatically to *ready for pickup* once all lines are resolved, since no packing step exists in a shop of this kind. The customer walks over, states the pickup code, pays in person, and the merchant marks the order complete. If the merchant never responds, the expiry job moves the order to `EXPIRED` rather than leaving it pending indefinitely.

Step 7 — Confidence decays. An hourly job examines availability records whose timestamps have aged past the configured thresholds and decays them towards `USUALLY_AVAILABLE`, writing `AUTO_DECAY` events. This is what prevents a single confirmation from being displayed as fresh indefinitely.

3.4 Project Modules

The system decomposes into nine functional modules.

1. **Authentication and identity.** Customer login by phone and one-time password; merchant and administrator login by password with Argon2 hashing; JWT access and refresh token issuance; role-based route guards. The same phone number may hold both a customer and a merchant account, so uniqueness is enforced on the pair of phone and role rather than on the phone alone.
2. **Merchant registration.** A seven-step wizard covering owner details, shop details, map location, opening hours, fulfilment options, document upload and starter inventory selection. Progress is preserved in browser storage between steps; the final submission creates the user, shop, address and initial inventory atomically in a single transaction, leaving the shop in `PENDING` status awaiting approval.
3. **Shop and inventory management.** Shop profile and opening-hours editing, bulk catalogue addition against the global product master, price setting, barcode-assisted lookup and inventory copying from a comparable shop.
4. **Discovery and search.** Radius-scoped nearby-shop listing, single-item search with typo tolerance, composite ranking, and category browsing.
5. **Multi-item search.** Shopping-list coverage analysis producing a best single shop and a greedy two-shop split (Section 3.11).
6. **Availability and confidence.** Badge derivation from state and timestamp, confidence scoring for ranking, the hourly decay job and the append-only event log.
7. **Orders and fulfilment.** Cart, checkout, the state machine of Section 3.12, per-line fulfilment status, pickup codes, automatic expiry and cancellation.
8. **Reviews and disputes.** One review per completed order with automatic recalculation of the shop's aggregate rating; structured dispute raising with an administrative resolution queue.

9. **Administration and analytics.** Shop moderation, catalogue oversight, order exploration, dispute resolution, live ranking-weight and decay-threshold controls, and analytics including unmet-demand reporting derived from zero-result searches.

3.5 System Architecture

Figure 3.1 shows the runtime architecture. The dashed boundary marks the process running on the server; everything above it runs in a browser. Note that all three clients enter through the same middleware pipeline, so a security control such as role enforcement is written once and cannot be bypassed by any client.

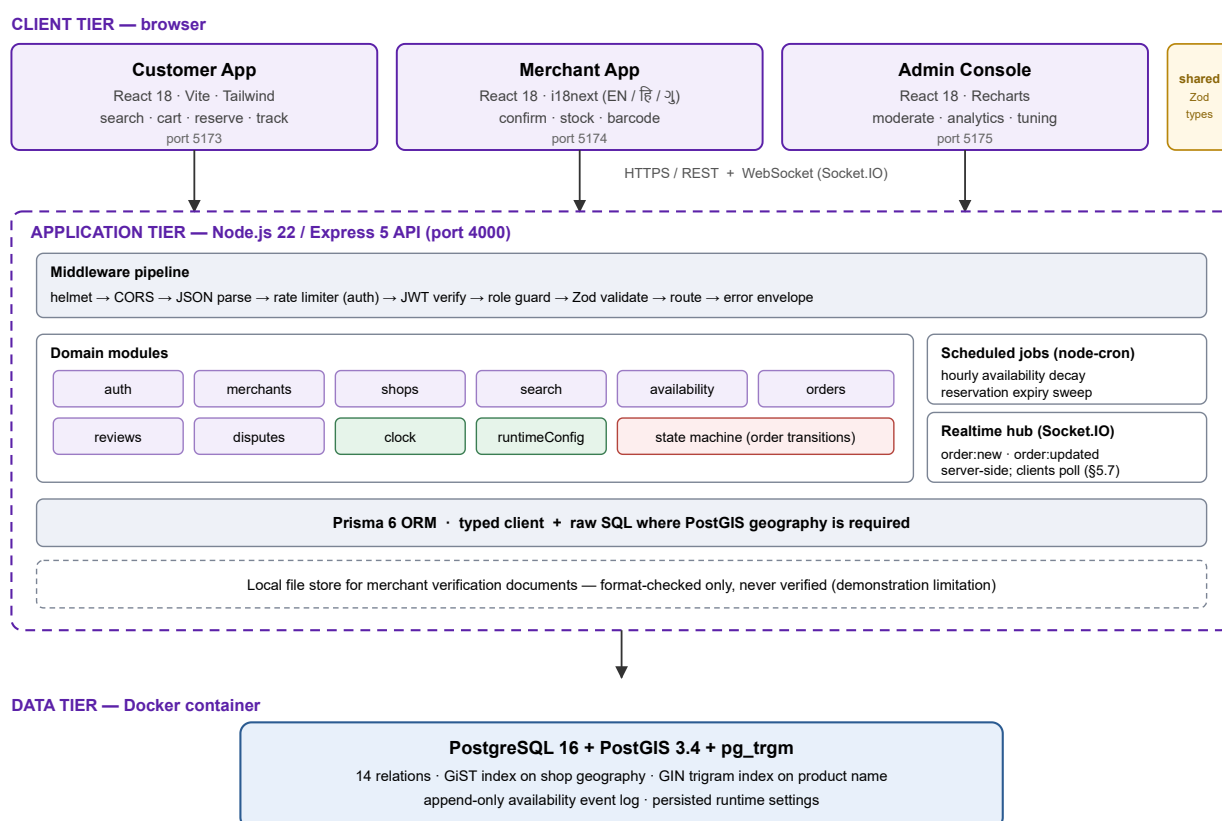


Fig. 3.1 — System architecture of ShopNear

3.6 Use Case Diagram

Figure 3.2 identifies the three actors and the use cases each may invoke. Role separation is enforced in middleware, not merely in the interface: a customer token presented to a merchant endpoint is rejected with HTTP 403, and one merchant cannot read another merchant’s orders.



Fig. 3.2 — Use case diagram

3.7 Data Flow Diagrams

3.7.1 Level 0 — Context diagram

At the highest level the system is a single process mediating between three external actors. Figure 3.3 shows what each actor supplies and what each receives.

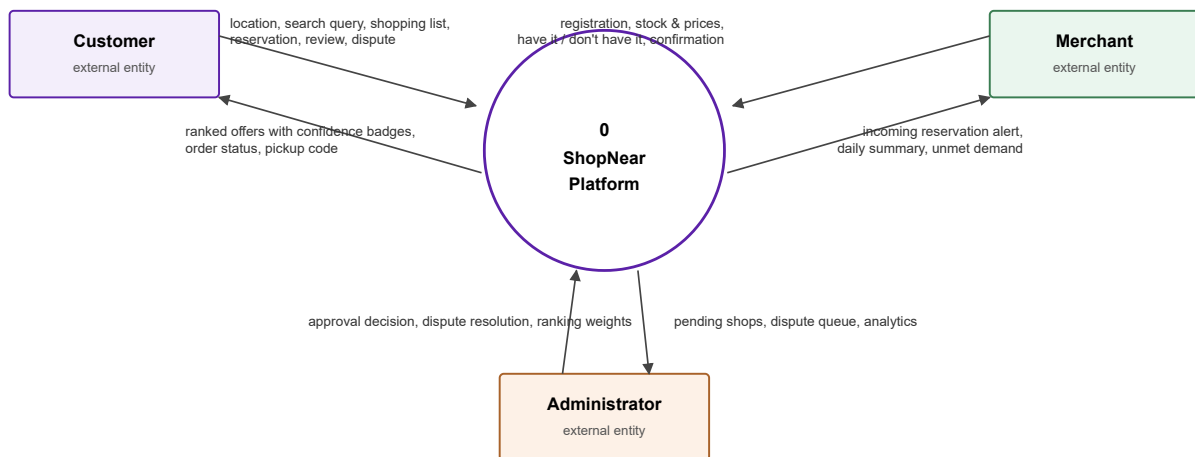


Fig. 3.3 — Data flow diagram, Level 0 (context diagram)

3.7.2 Level 1 — Major processes and data stores

Figure 3.4 decomposes the context process into six sub-processes and five data stores. The flow worth following carefully is the one from process 4.0: a merchant's answer to an incoming reservation updates the order *and* writes both an availability record and an audit event, which is how a single merchant action improves the data every later customer sees.

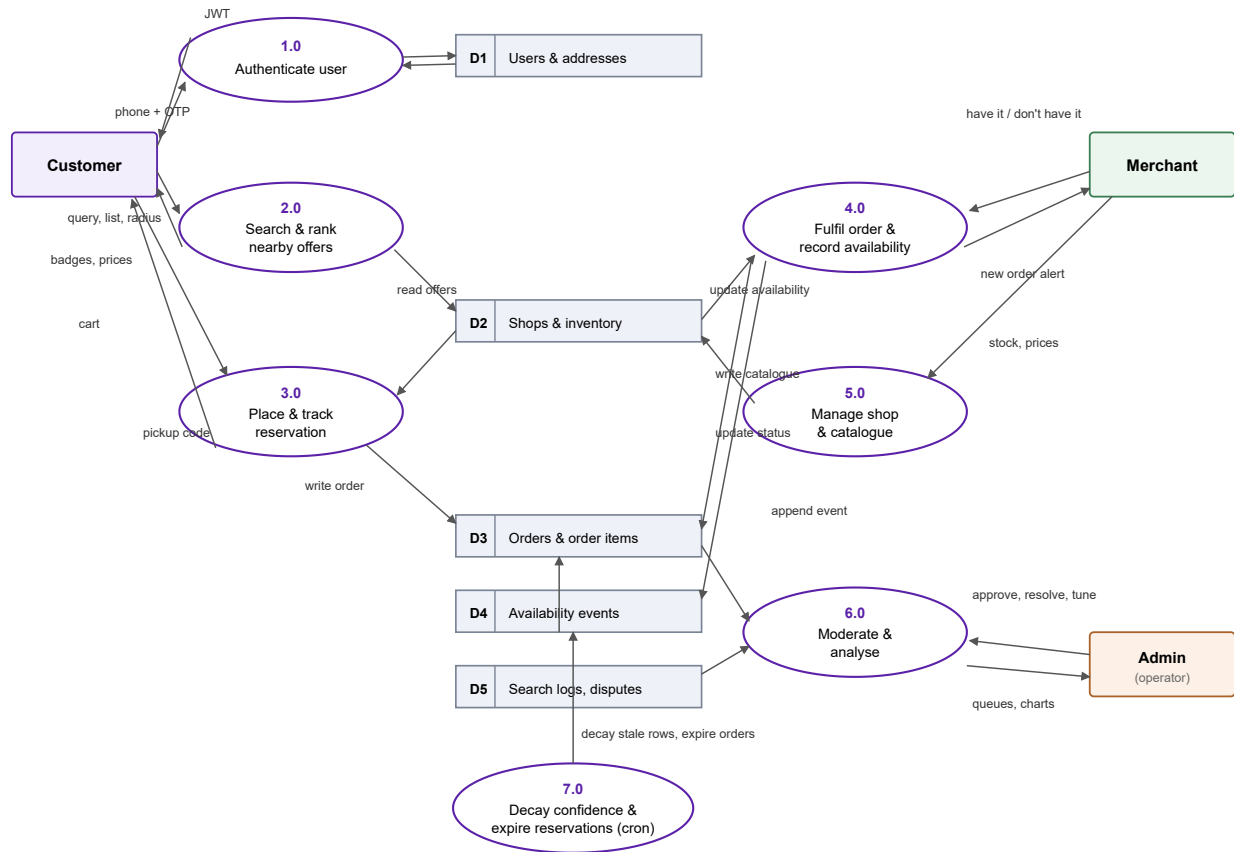


Fig. 3.4 — Data flow diagram, Level 1

3.8 Entity-Relationship Diagram

The schema comprises fourteen relations. Figure 3.5 shows the eleven that carry the core business relationships; the three supporting relations — `SearchLog`, `AvailabilityEvent` and `AppSetting` — are shown detached because they are written to but never joined against during a customer request.

Three modelling decisions deserve attention. First, the `Product`–`ShopInventory`–`Shop` chain is the associative pattern that makes cross-shop comparison possible: `ShopInventory` is a full entity with its own attributes (price, availability, timestamp), not a bare junction table. Second, `OrderItem` stores snapshot copies of the product name, unit label and unit price rather than relying on the live `Product` row, so that a historical order remains an accurate record even after the catalogue changes. Third, `Shop.isOpenNow` is deliberately *not* a column; it is computed at read time from the stored opening hours and the current instant, because storing it would guarantee it was wrong between updates.

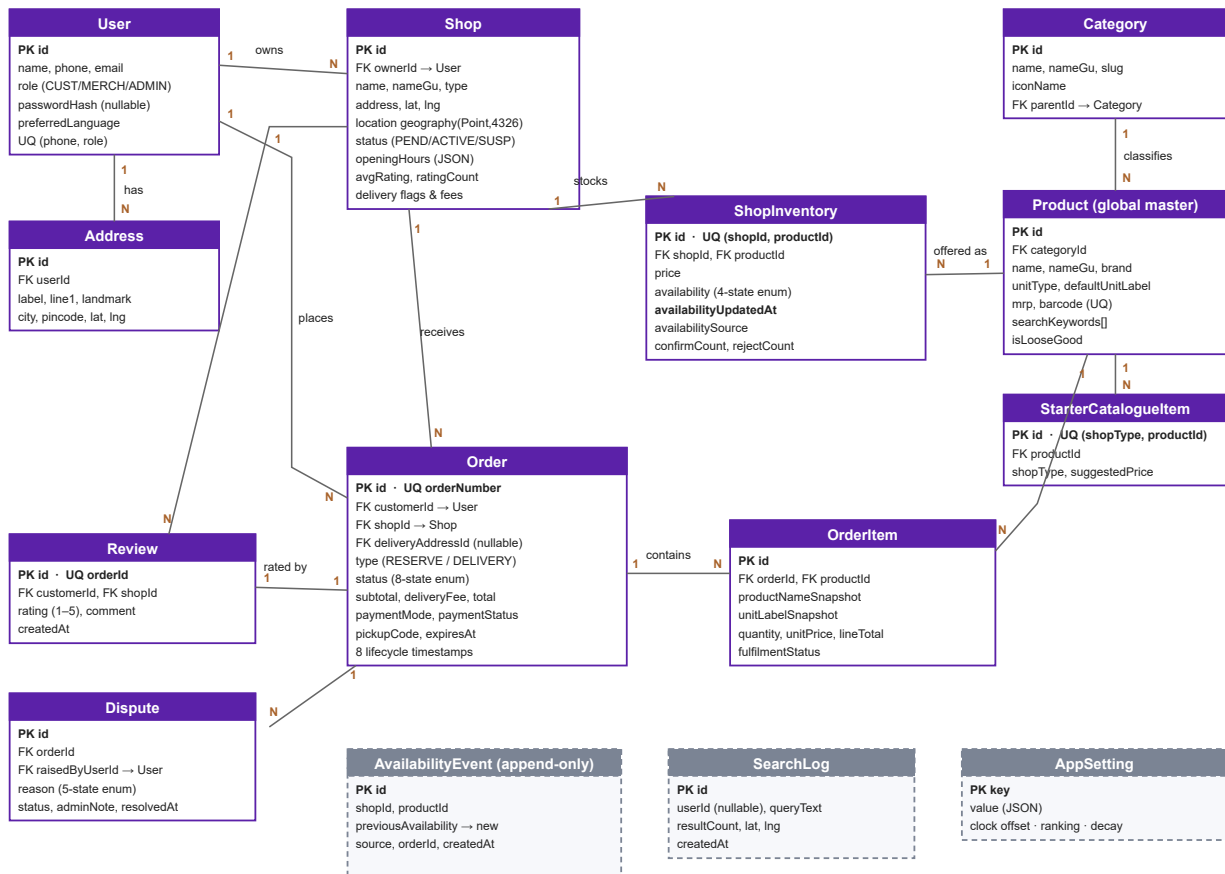


Fig. 3.5 — Entity-relationship diagram

3.9 The Availability Confidence Model

This section describes the central contribution of the project.

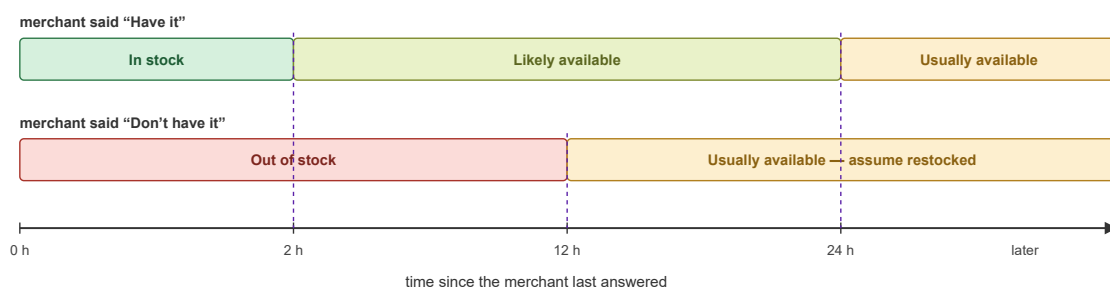
Each `ShopInventory` row stores an availability state drawn from four values — `IN_STOCK`, `OUT_OF_STOCK`, `USUALLY_AVAILABLE`, `UNKNOWN` — together with the instant at which that state was last set and its provenance. The label shown to the customer is not the stored state. It is *derived* from the state and the age of the timestamp, evaluated against the current instant supplied by the clock service. Table 3.2 gives the complete rule set.

Stored state	Age of timestamp	Badge displayed	Rationale
<code>IN_STOCK</code>	under 2 hours	In stock (green) “confirmed 20 min ago”	Recent positive confirmation; the exact age is stated so the customer can judge it themselves
<code>IN_STOCK</code>	2 to 24 hours	Likely available (green-amber)	Still probably true, but no longer a fresh observation

Stored state	Age of timestamp	Badge displayed	Rationale
IN_STOCK	over 24 hours	Usually available (amber)	The signal has aged into a general statement about the shop rather than a current fact
USUALLY_AVAILABLE	any	Usually available (amber) “this shop normally stocks this”	A statement about the shop’s typical range, not about today
OUT_OF_STOCK	under 12 hours	Out of stock (red)	A recent negative signal is trusted
OUT_OF_STOCK	over 12 hours	Usually available (amber)	Restocking is assumed rather than distrusting an old negative indefinitely
UNKNOWN or no record	—	Ask the shop (grey)	Absence of data is stated plainly; reservation is still permitted

Table 3.2 — Availability badge derivation rules

Two rows in this table are counter-intuitive and were arrived at deliberately. A *stale negative* becomes “usually available” rather than remaining “out of stock”: a shop that lacked an item yesterday has very likely restocked, and continuing to display it as unavailable would suppress a shop that can in fact serve the customer. And an *unknown* state does not hide the shop or block a reservation — a customer who is willing to walk over and ask should not be prevented from doing so by the platform’s ignorance.



Thresholds are runtime configuration, adjustable from the admin console without redeployment

Fig. 3.6 — Availability confidence decay over time

An hourly scheduled job walks records whose timestamps have crossed these thresholds and rewrites the stored state towards `USUALLY_AVAILABLE`, recording each change in the append-only event log with source `AUTO_DECAY`. Derivation alone would be sufficient for display; the job exists

so that the decay is materialised, auditable and visible to the ranking function, and so that the event log constitutes a complete history of how each availability signal evolved.

3.10 Geospatial Search and Composite Ranking

Search proceeds in three stages: *spatial narrowing*, *textual matching*, then *composite ranking*.

Spatial narrowing uses PostGIS. Each shop carries a `geography(Point, 4326)` column kept in step with its latitude and longitude, indexed with GiST. A radius query is expressed as `ST_DWithin(location, point, radius)`, which computes true distance on the ellipsoid and is answered using the index. The alternative — fetching all shops and computing the haversine formula in application code — cannot use an index and degrades linearly as the shop count grows.

Textual matching first tests the query against the product name and the curated `searchKeywords` array, which holds transliterations deliberately seeded for this purpose, so that “atta”, “aata”, “आटा” and “ऐलट” all reach wheat flour. Where no keyword matches, trigram similarity over a GIN index supplies typo tolerance, recovering matches for inputs such as “ata” or “attta”.

Ranking is then a weighted composite rather than distance alone:

```
score = w1·availabilityConfidence + w2·proximity + w3·shopRating + w4·isOpenNow

where  availabilityConfidence = badge tone mapped to [0, 1]
       proximity              = 1 - min(distance / radius, 1)
       shopRating             = clamp(avgRating, 0, 5) / 5
       isOpenNow              = 1 if currently open, else 0
```

Component	Default weight	Reason for the weighting
Availability confidence	0.40	Highest, because a nearby shop that does not have the item is of no use; usefulness dominates closeness
Proximity	0.30	The reason the customer opened the application, but subordinate to actually having the item
Shop rating	0.20	A tie-breaker reflecting past service quality
Open now	0.10	A closed shop is still worth showing, since the customer may be planning ahead

Table 3.3 — Default composite ranking weights

These weights are held in a persisted settings row rather than compiled into the code, and are exposed as sliders in the administrator console. Adjusting a weight visibly reorders live search results, which makes the ranking function inspectable rather than opaque. The computed score is used solely for ordering and is never transmitted to any client.

3.11 Multi-Item Search Algorithm

Households shop from lists, not from single items. Given a list such as *atta, doodh, Maggi, sabun*, the useful question is not where each item is individually cheapest but **which single nearby shop can supply the most of the list**, since each additional shop visited is a real cost in time. This is a variant of the maximum-coverage problem, which is NP-hard in general; the greedy approximation is used here, which is both standard and appropriate given that the candidate set is bounded by the search radius.

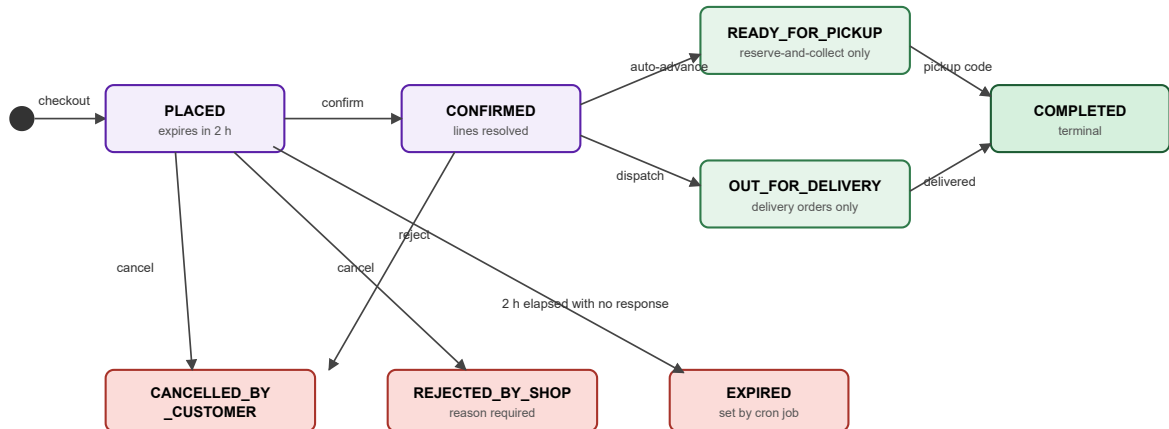
```
Input   : list of item terms L, customer location P, radius r
Output  : best single shop, plus a two-shop split for the remainder

1  for each item i in L
2      C[i] ← findCandidates(i, P, r)      // same matcher as single-item search
3      if C[i] is empty then log a zero-result SearchLog entry for i
4
5  coverage ← map from shop → set of items that shop can supply
6  for each item i, for each shop s in C[i] : coverage[s] ← coverage[s] ∪ {i}
7
8  best ← argmax over s of ( |coverage[s]| , -distance(s) )  // ties broken by
nearness
9  missing ← L \ coverage[best]
10
11 if missing is non-empty
12     second ← argmax over s ≠ best of ( |coverage[s] ∩ missing| , -distance(s) )
13     return { best, split: [best, second] }
14 return { best, split: [best] }
```

Two design details matter. Ties in coverage are broken by proximity, so that where two shops cover equally many items the nearer one wins. And every item that no shop within the radius can supply is logged individually as a zero-result search — not merely the query as a whole — so that the administrator’s unmet-demand report can name the exact product the neighbourhood is failing to supply. That report is arguably more valuable to a shopkeeper than any completed order, because it identifies revenue currently being lost.

3.12 Order State Machine

An order is the only object in the system whose incorrect handling costs someone money or a wasted journey, so its lifecycle is governed by an explicit state machine rather than by scattered conditional logic. Every status change in the service layer is routed through a single validation function; no code writes an order status directly. An illegal transition raises an error and is rejected at the domain layer, before any database write occurs.



Every arrow shown is legal; every arrow not shown is rejected with HTTP 400 before any write occurs.
26 automated tests exercise this table, including each illegal transition and each per-type restriction.

Fig. 3.7 — Order state machine

Current state	Permitted next states
PLACED	CONFIRMED , REJECTED_BY_SHOP , CANCELLED_BY_CUSTOMER , EXPIRED
CONFIRMED	READY_FOR_PICKUP , OUT_FOR_DELIVERY , CANCELLED_BY_CUSTOMER
READY_FOR_PICKUP	COMPLETED
OUT_FOR_DELIVERY	COMPLETED
COMPLETED , CANCELLED_BY_CUSTOMER , REJECTED_BY_SHOP , EXPIRED	none — terminal

Table 3.4 — Legal order state transitions

The type restriction is enforced within the same function rather than only in the service layer: `READY_FOR_PICKUP` is reachable only by a reserve-and-collect order and `OUT_FOR_DELIVERY` only by a delivery order, so it is not possible to construct a call that violates this even though both states appear as successors of `CONFIRMED` in the table. The automatic advance from `CONFIRMED` to `READY_FOR_PICKUP` once all lines are resolved reflects the reality of the shop: there is no packing stage between confirming an item and setting it on the counter, and inserting one would have added a second interaction to a workflow deliberately designed around a single tap.

CHAPTER IV

SYSTEM REQUIREMENTS

4.1 Software Requirements

Every dependency listed below is free and open source, and the complete system runs on a single machine with no external service and no internet connection after the initial installation. This was a deliberate constraint: a project that cannot be demonstrated without a network is a project that cannot be reliably demonstrated.

Category	Tool / technology	Purpose
Operating system	Windows 10/11, macOS 12+, or Ubuntu 20.04+	Development and demonstration host
Programming language	TypeScript 5.6	Single language across server, all three clients and shared contracts
Runtime	Node.js 22 LTS	Executes the API server, build tooling and test runner
Package manager	npm 10 (workspaces)	Monorepo dependency management and cross-package linking
Web framework (server)	Express 5	REST routing and middleware pipeline
Database	PostgreSQL 16	Primary relational data store
Spatial extension	PostGIS 3.4	Geodesic radius queries with GiST spatial indexing
Text-search extension	pg_trgm	Trigram similarity for typo-tolerant product search
ORM and migrations	Prisma 6	Schema definition, versioned migrations, type-safe query client
Containerisation	Docker / Docker Compose	Reproducible PostGIS instance without host installation
Frontend framework	React 18	Component model for all three client applications
Build tool	Vite 5	Development server with hot module replacement; production bundling
Styling	Tailwind CSS 3, PostCSS	Utility-first design system shared across the clients
Client routing	React Router 6	Declarative routes and role-based route guards

Category	Tool / technology	Purpose
Server-state management	TanStack Query 5	Caching, revalidation and request de-duplication
Authentication	jsonwebtoken 9, argon2	Access and refresh token issuance; password hashing
Security middleware	helmet, cors, express-rate-limit	Response headers, cross-origin policy, brute-force limiting
Realtime transport	Socket.IO 4	Authenticated event channel on the API, with room-scoped order events
Validation	Zod	Request schema validation shared between client and server
Scheduling	node-cron	Hourly decay job and reservation expiry sweep
File upload	multer	Local storage of merchant verification documents
Internationalisation	i18next, react-i18next	English, Hindi and Gujarati translation of the merchant console
Barcode input	@zxing/browser	Camera-based barcode reading with manual fallback
Charting	Recharts 2	Analytics visualisation in the administrator console
Testing	Vitest 2, Supertest 7	Unit and HTTP integration testing
Development environment	Visual Studio Code, Git	Editing, debugging and version control

Table 4.1 — Software requirements

4.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core x86-64, 2.0 GHz (Intel Core i3 or Apple M1)	Quad-core (Intel Core i5/i7 or Apple M2/M3) — the database container and three development servers run concurrently
Memory	8 GB RAM	16 GB — PostgreSQL container plus three Vite processes and the API
Storage	5 GB free (SSD preferred)	20 GB SSD — accommodates <code>node_modules</code> , the Docker image and the database volume

Component	Minimum	Recommended
Virtualisation	Hardware virtualisation enabled in BIOS/UEFI	Required by Docker Desktop for the PostGIS container
Display	1366 × 768	1920 × 1080 — the administrator console assumes a desktop-width viewport
Mobile device (optional)	Any smartphone with a modern browser	Device with a camera, for barcode scanning in the merchant console
Network	Required for first-time installation only	Broadband for the initial <code>npm install</code> and Docker image pull; the system runs fully offline thereafter

Table 4.2 — Hardware requirements

4.3 Functional Requirements

FR1 — Authentication. Customers shall authenticate by phone number and one-time password. Merchants and administrators shall authenticate by password. The system shall issue a short-lived access token and a longer-lived refresh token. A single phone number shall be permitted to hold one customer account and one merchant account concurrently.

FR2 — Merchant registration. A prospective merchant shall complete a multi-step registration wizard capturing owner details, shop details, map location, opening hours, fulfilment preferences, verification documents and an initial catalogue. Progress shall survive navigation between steps. Submission shall create all records atomically and place the shop in a pending state awaiting approval.

FR3 — Location-scoped discovery. The customer application shall require a location before displaying any catalogue, and shall support radius selection from 250 m, 500 m, 1 km and 3 km.

FR4 — Item search. The system shall return products available within the selected radius, matching on product name, curated keywords and trigram similarity, ordered by the composite ranking score.

FR5 — Availability display. Every offer shall display a confidence badge derived from its availability state and the age of its timestamp. The system shall never display a numeric stock quantity in the customer interface.

FR6 — Multi-item search. Given a list of item terms, the system shall identify the single shop covering the greatest number of items and propose a two-shop split for the remainder.

FR7 — Reservation. A customer shall place a reservation against one shop, receive a pickup code, and track status changes without refreshing the page.

FR8 — Merchant fulfilment. A merchant shall be alerted to an incoming reservation, mark each line available or unavailable, and confirm or reject the order. Each such answer shall update that shop's availability record and append an audit event.

FR9 — Order lifecycle integrity. Every status change shall be validated against the transition table of Table 3.4. Illegal transitions shall be rejected. Unanswered reservations shall expire automatically after the configured interval.

FR10 — Inventory management. A merchant shall add products in bulk from the global catalogue, set prices, look products up by barcode, and copy an initial inventory from a comparable shop.

FR11 — Reviews and disputes. A customer shall review a completed order once, updating the shop's aggregate rating, and shall raise a structured dispute for administrative resolution.

FR12 — Administration. An administrator shall moderate shops, resolve disputes, explore orders, adjust ranking weights and decay thresholds at runtime, and view analytics including unmet-demand reporting.

4.4 Non-Functional Requirements

Requirement	Specification	Implementation
Performance	Search results returned in under 300 ms on the seeded dataset	GiST spatial index on shop geography; GIN trigram index on product name; composite indexes on the inventory and order tables
Responsiveness	Customer and merchant applications usable at 390 × 844 px	Mobile-first layouts with bottom navigation; desktop treated as the secondary target
Internationalisation	English, Hindi and Gujarati	Full i18next coverage of the merchant console with an in-header language switch; Gujarati names stored alongside English for shops, categories and products
Accessibility	Availability must never be conveyed by colour alone	Every badge carries a colour, a text label and an icon; semantic markup, keyboard navigation and visible focus rings throughout
Security	Demonstrable hygiene appropriate to a non-production system	Argon2 password hashing, JWT access and refresh tokens, role middleware, cross-tenant ownership checks, Zod validation on every endpoint, rate limiting on authentication routes, security headers via helmet, no secrets committed to version control

Requirement	Specification	Implementation
Reliability	Consistent error handling; no partial writes	Single error envelope { error: { code, message, details } } across all endpoints; multi-record operations wrapped in database transactions; no stack traces exposed to clients
Maintainability	Domain logic isolated and independently testable	Route, schema and service layers separated per module; confidence, ranking and state-machine logic implemented as pure functions with no database dependency
Reproducibility	Identical demonstration dataset on every machine	Deterministic seeded pseudo-random generator; two resets produce byte-identical content, verifiable by content hash
Portability	Full offline operation after installation	Database supplied as a Docker container; no paid or network-dependent third-party service in any code path
Testability	Automated coverage of all domain rules	379 automated tests across 35 files, executable with a single command

Table 4.3 — Non-functional requirements and how each is met

CHAPTER V

IMPLEMENTATION AND RESULTS

5.1 Introduction

This chapter documents what was built, layer by layer, and presents the resulting interfaces. Every screenshot reproduced here was captured from the application running on a development machine against the seeded dataset; none is a mock-up or a design comp.

The delivered system comprises approximately 29,000 lines of TypeScript. Table 5.3 gives the distribution across the codebase. The whole system starts from a clean checkout with four commands: copy the environment file, start the database container, install dependencies, reset and seed the database.

5.2 Database Layer

The schema is defined declaratively in Prisma and applied through six versioned migrations. The first migration provisions the PostGIS and `pg_trgm` extensions; subsequent migrations build identity and geography, the catalogue, the order and dispute tables, the specialised indexes, and the runtime settings store.

Two constructs required care because Prisma has no native representation for them. The `geography(Point, 4326)` column is declared as an unsupported type and written through raw SQL, kept in step with the shop's latitude and longitude by the seed and the shop service. The GiST and GIN indexes are declared in the schema so that Prisma owns them and they are recreated by any migration reset, rather than being applied by hand and silently lost.

The seed generator is **deterministic**. A single seeded pseudo-random generator is threaded through every stage, so two consecutive resets produce identical content — the same products, the same prices, the same shops and the same availability states. This is verifiable by computing a content hash over the inventory join after two separate resets. Determinism matters for a demonstration: the presenter knows exactly what will appear, and a defect can be reproduced rather than merely described.

Seeded entity	Count	Composition
Products (global catalogue)	342	Across nine trade categories, each with brand, unit, barcode and transliterated search keywords
Shops	14	11 active, 2 pending approval, 1 suspended — so that the moderation states are demonstrable, not hypothetical
Shop inventory records	1,749	Priced per shop, with a realistic spread of availability states and timestamp ages

Seeded entity	Count	Composition
Historical orders	120	Distributed across statuses, with line items, reviews and open disputes

Table 5.1 — Seeded demonstration dataset

The seeded shops are drawn from seven trade types — kirana, general store, stationery, hardware, chemist, bakery and dairy — positioned around a fixed anchor point in Navrangpura, Ahmedabad, at distances from 80 m to several kilometres. This spread is what makes the radius selector meaningful: changing from 250 m to 3 km visibly changes the result set.

5.3 Backend API Development

The API is an Express 5 application organised by domain module. Each module contributes a route file, a Zod schema file and a service file, so that transport, validation and business logic remain separable and the service layer stays independently testable.

Every request passes through the same middleware pipeline: security headers, cross-origin policy, body parsing, rate limiting on authentication routes, token verification, role enforcement, schema validation, the route handler, and finally a single error handler that converts any thrown error into the uniform envelope `{ error: { code, message, details } }`. Because the pipeline is shared, a control such as role enforcement is written once and cannot be bypassed by any of the three clients.

Module	Method	Path	Purpose
Authentication	POST	<code>/api/auth/otp/request</code>	Request a customer one-time password
	POST	<code>/api/auth/otp/verify</code>	Verify the code and issue tokens
	POST	<code>/api/auth/customer/profile</code>	Complete a new customer profile
	POST	<code>/api/auth/merchant/login</code>	Merchant password login
	POST	<code>/api/auth/admin/login</code>	Administrator password login
	POST	<code>/api/auth/refresh</code>	Exchange a refresh token for a new access token
	GET	<code>/api/auth/me</code>	Return the authenticated principal

Module	Method	Path	Purpose
Merchants	POST	/api/merchants/register	Atomic creation of user, shop, address and starter inventory
	GET	/api/merchants/starter-catalogue	Curated starter products for a shop type
	POST	/api/merchants/uploads	Store a verification document (format check only)
Shops	GET	/api/shops/nearby	Active shops within a radius, with computed opening status
	GET	/api/shops/:id	Shop detail
	PATCH	/api/shops/:id	Update profile and opening hours (owner only)
	GET	/api/shops/:id/inventory	Shop catalogue with confidence badges
	PUT	/api/shops/:id/inventory	Bulk inventory and price update
	POST	/api/shops/:id/inventory/copy-from/:otherShopId	Copy inventory from a comparable shop
Search	GET	/api/search	Single-item search with ranking and confidence
	POST	/api/search/multi	Shopping-list coverage and two-shop split

Module	Method	Path	Purpose
Orders	POST	/api/orders	Place a reservation or delivery order
	GET	/api/orders/mine	Customer order history
	GET	/api/orders/shop/:shopId	Shop order list (owner only)
	GET	/api/orders/:id	Order detail
	POST	/api/orders/:id/confirm	Confirm with per-line fulfilment outcomes
	POST	/api/orders/:id/reject	Reject with a reason
	POST	/api/orders/:id/out-for-delivery	Dispatch a delivery order
	POST	/api/orders/:id/complete	Complete against the pickup code
	POST	/api/orders/:id/cancel	Customer cancellation
Reviews	POST	/api/orders/:orderId/reviews	Review a completed order; recalculates the shop rating
	GET	/api/shops/:shopId/reviews	Reviews for a shop
Disputes	POST	/api/orders/:orderId/disputes	Raise a structured dispute
	GET	/api/admin/disputes	Administrative dispute queue
	POST	/api/admin/disputes/:id/resolve	Resolve or reject a dispute
Health	GET	/health	Reports API, database and PostGIS status

Table 5.2 — REST API endpoints implemented

5.4 Customer Application

The customer application is mobile-first, laid out for a 390×844 px viewport with a bottom navigation bar, and reflows to a wider desktop layout above that breakpoint.

Location gate. The application will not display a catalogue until a location is chosen. This is a hard gate rather than a dismissible prompt, because every result the system produces is relative to a point. The customer may use device geolocation or select one of three demonstration positions at different distances from the shop cluster, which makes radius behaviour presentable without moving physically.

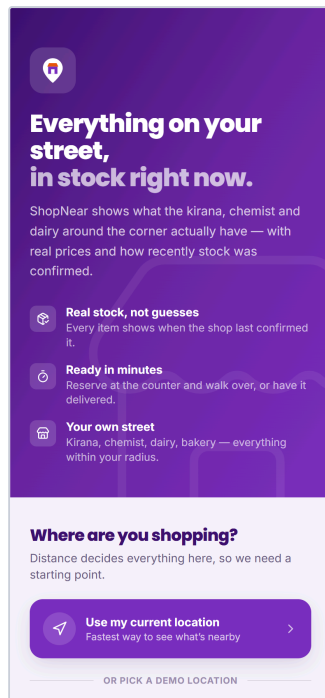


Fig. 5.1 — Location gate

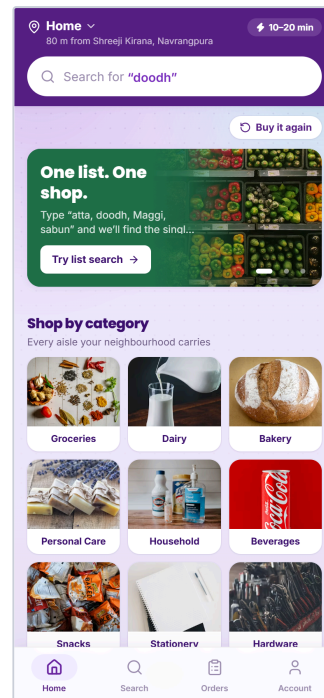


Fig. 5.2 — Neighbourhood home feed

Home feed. Once anchored, the home screen presents the search entry point with rotating placeholder terms in the vocabulary customers actually use, a promotional rail for the shopping-list feature, and a category grid covering every aisle the neighbourhood carries. The header states the active location and its relationship to a landmark shop — “80 m from Shreeji Kirana” — so the anchor point is never ambiguous.

Search results and confidence badges. Figure 5.3 shows the result of searching for *atta*. Each card carries the price, the discount against maximum retail price, the walking time, and the confidence badge. Critically, the badge is rendered as colour *and* text *and* icon, satisfying the accessibility requirement that availability never be conveyed by colour alone. The *Compare shops* control on each card opens a per-product comparison across every nearby shop carrying it — the view that makes the global-catalogue design decision of Section 3.1 pay off.

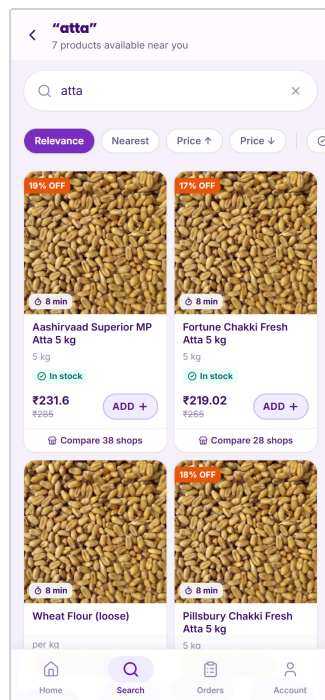


Fig. 5.3 — Search results with confidence badges

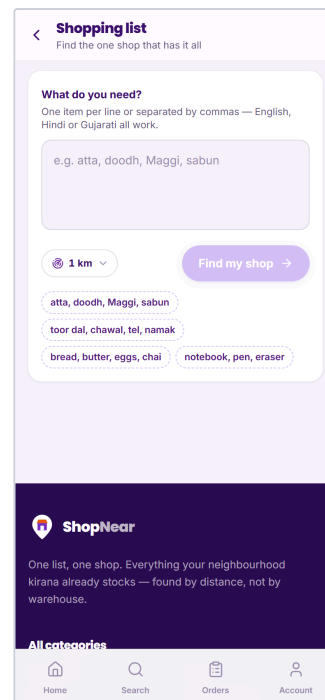


Fig. 5.4 — Multi-item shopping-list search

Multi-item search. The shopping-list screen accepts items one per line or comma-separated, in English, Hindi or Gujarati, and applies the coverage algorithm of Section 3.11. Preset example lists are offered so the feature can be demonstrated without typing. The result names the single shop covering the most items and, where the list cannot be completed at one shop, proposes a second.

Cart, checkout and tracking. The cart is scoped to a single shop, since a reservation is a commitment by one shopkeeper. Checkout captures the fulfilment type and payment mode, and issues a four-digit pickup code. The order screen then tracks status changes pushed over the realtime channel, without polling or a manual refresh.

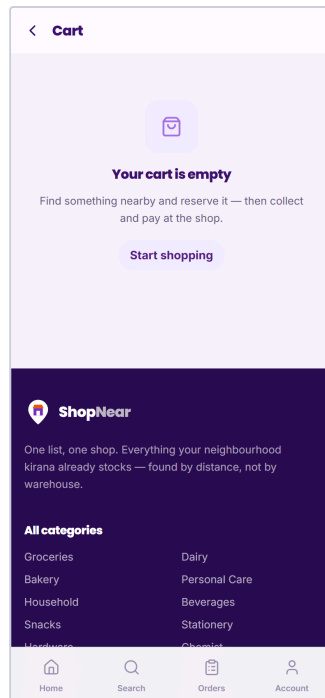


Fig. 5.5(a) — Cart

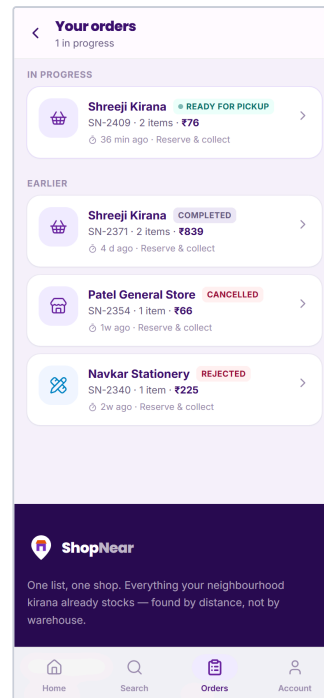


Fig. 5.5(b) — Order history

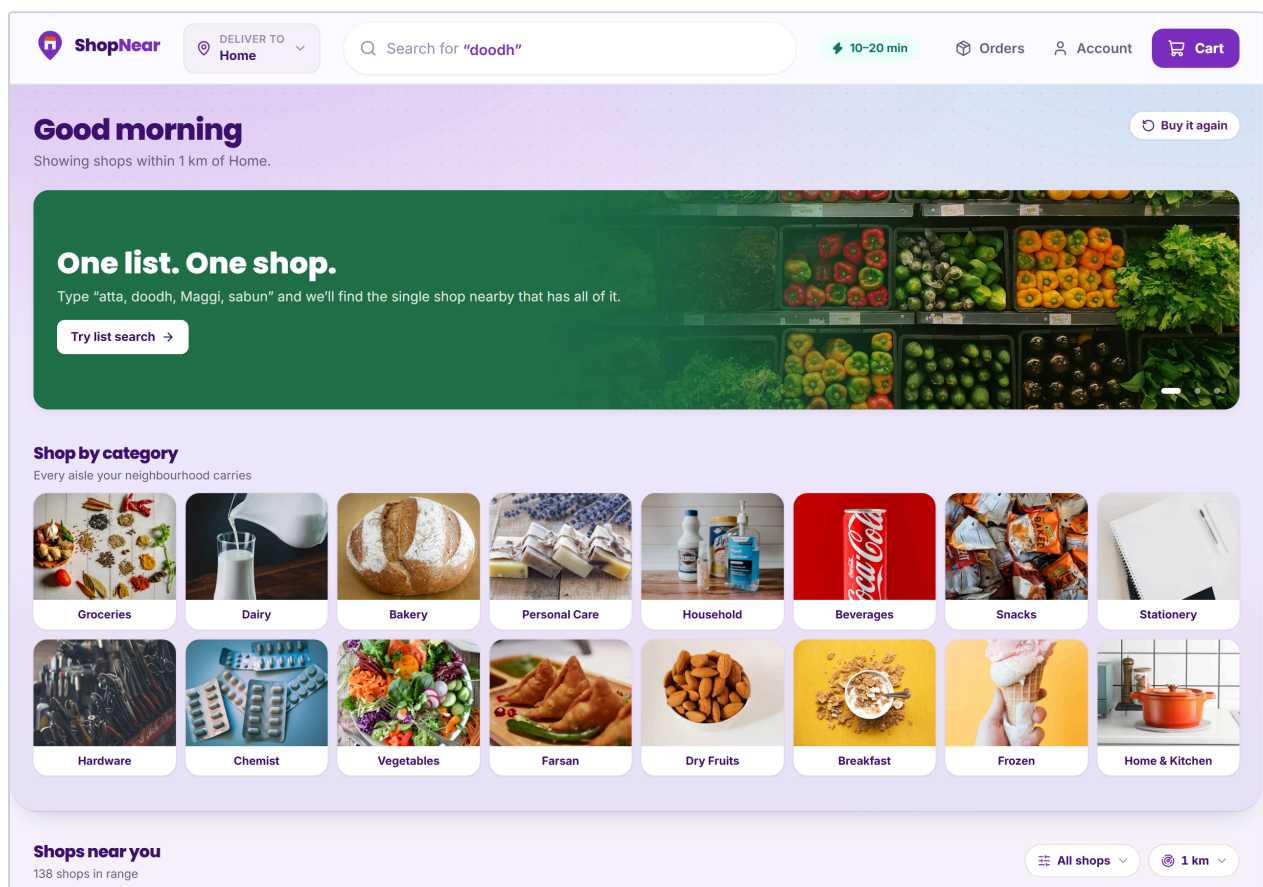


Fig. 5.6 — The same application at desktop width

5.5 Merchant Application

The merchant console is the interface with the strictest design constraints in the project. Its user is standing behind a counter, may be serving a customer at the same time, and may read Gujarati more comfortably than English. Every screen is therefore built around large targets, minimal text and a language switch that is visible at all times in the header.

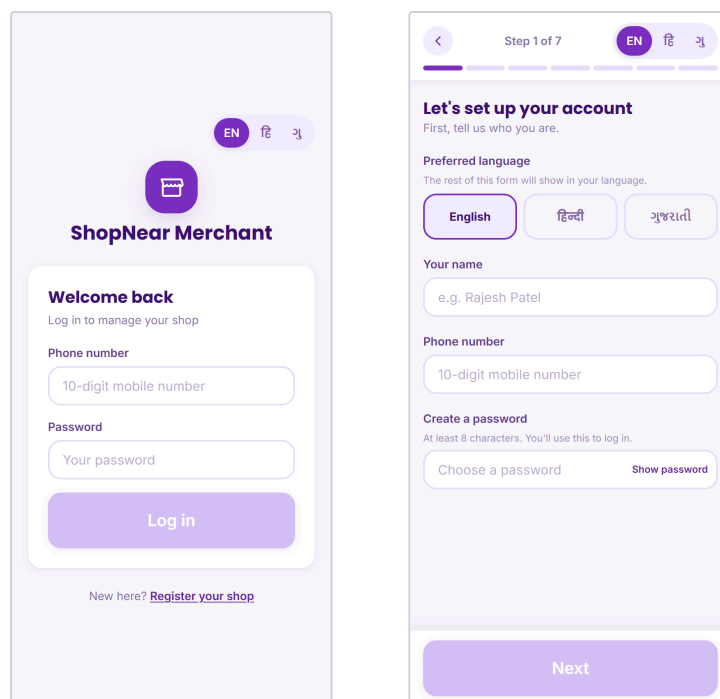


Fig. 5.7(a) — Merchant login Fig. 5.7(b) — Registration wizard

Registration. The seven-step wizard collects owner details, shop details, map position, opening hours, fulfilment preferences, verification documents and starter inventory. Progress is preserved in browser storage between steps, so a mistake or an interruption does not discard the work. Submission creates every record in one transaction and routes the merchant to a pending-approval screen; the shop becomes discoverable the moment an administrator approves it.

Daily summary. The dashboard answers the questions a shopkeeper actually has: how many reservations arrived today, how many were confirmed, how many were rejected, and what they were worth. Beneath these it surfaces the single most useful piece of intelligence the platform can offer — the item most often marked *Don't have it*. In Figure 5.8 this reads “Amul Processed Cheese Slices 200 g — said unavailable 5 times”: demand the shop is repeatedly failing to capture, quantified.

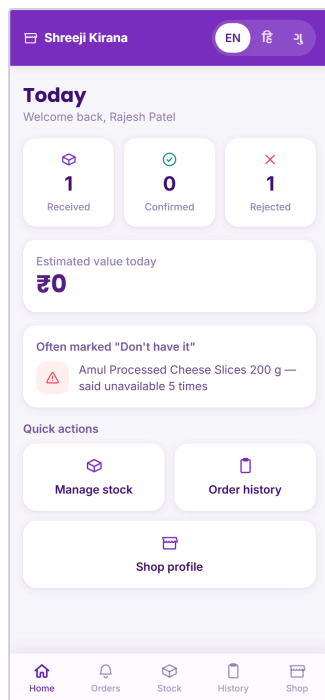


Fig. 5.8 — Daily summary dashboard

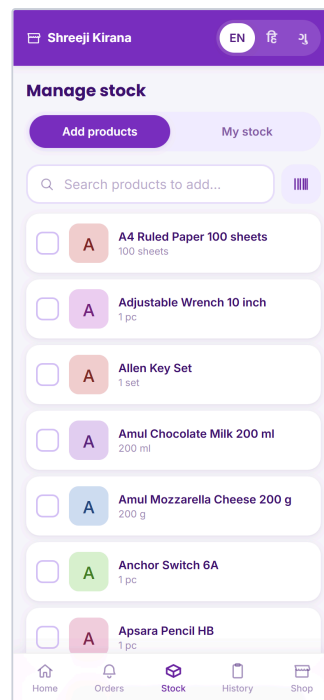


Fig. 5.9 — Bulk stock management

Incoming reservations and stock management. The incoming-reservation screen presents each order as a full-screen card with large *Have it* and *Don't have it* controls per line and a single confirm action, accompanied by an audible alert and a live countdown to expiry. The target for a complete response is under eight seconds. Stock management works against the global catalogue: the merchant searches, ticks products and sets prices in one pass, or uses the barcode control to add a product by scanning it.

Language. Figure 5.10 shows the same dashboard rendered in Gujarati. Translation covers navigation, labels, status vocabulary and summary text throughout the console. Product names fall back to English where no translation exists, which is correct behaviour rather than a gap — a brand name is not translated in the shop either.



Fig. 5.10 — The merchant console in Gujarati (ગુજરાતી)

5.6 Admin Console

The administrator console is a desktop-width operator tool covering shop moderation, catalogue oversight, order exploration, dispute resolution, analytics and live tuning of the ranking and decay parameters.

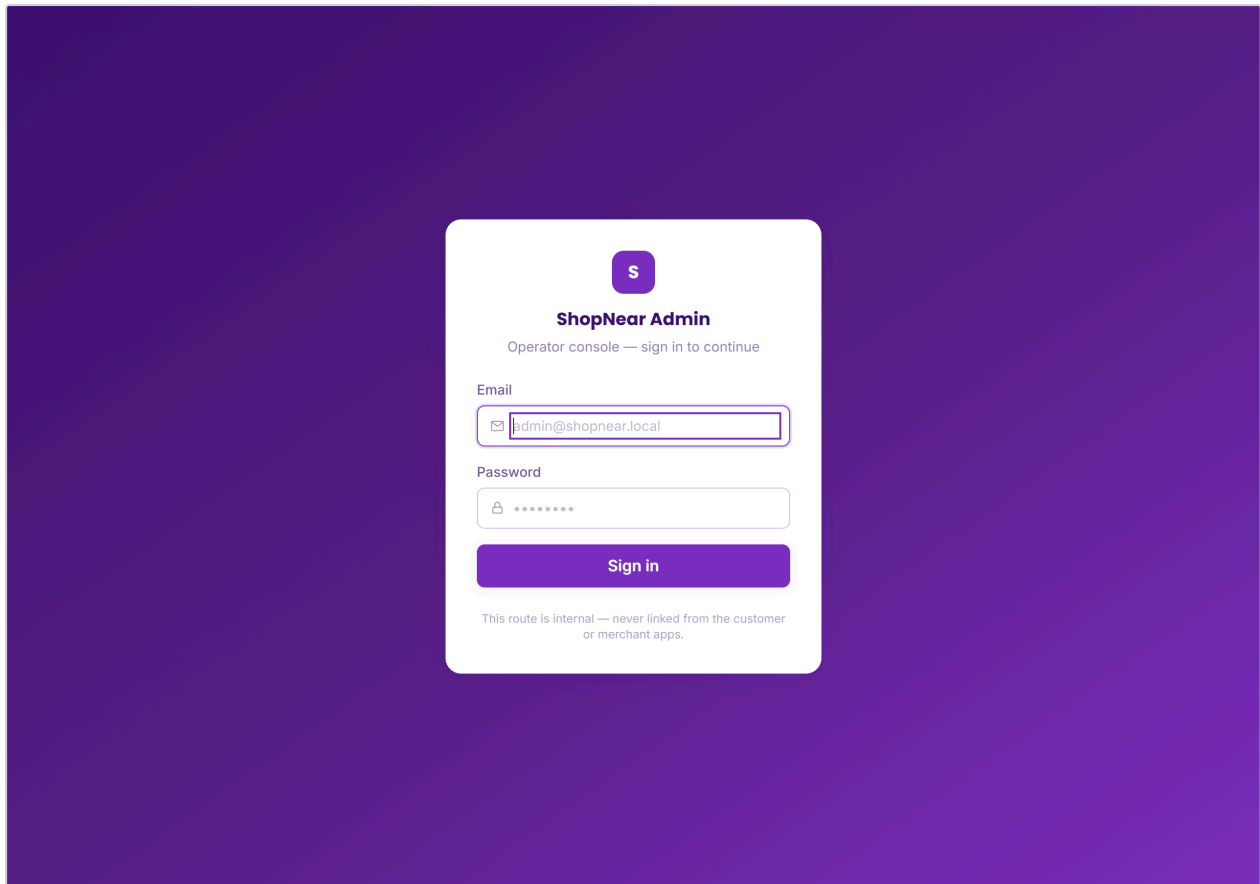


Fig. 5.11 — Administrator login

Analytics. Figure 5.12 shows the analytics dashboard, which reports active shops within a radius, the spread of trade types represented, and the open and resolved dispute counts, with charts drawn from live API responses. The page is deliberately explicit about provenance: its subtitle states that everything shown is either live data or an honestly labelled gap, and panels whose underlying endpoint does not yet exist say exactly that rather than displaying invented figures. This is discussed further in Section 7.3.

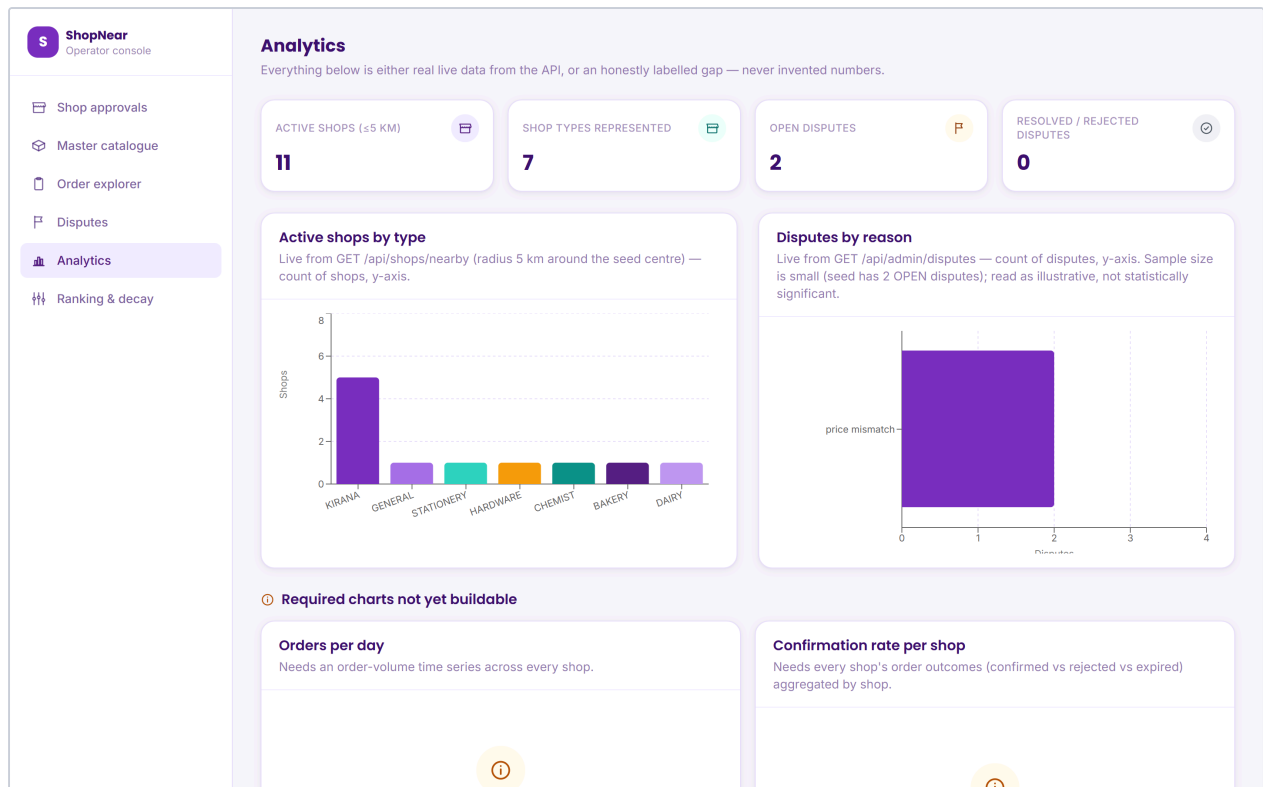


Fig. 5.12 — Analytics dashboard

Ranking and decay controls. Figure 5.13 is the most direct demonstration of the algorithms described in Chapter III. The page runs a live search against the same endpoint the customer application uses, then exposes the four ranking weights and the three decay thresholds as sliders. Moving a weight visibly reorders the results; moving a decay threshold changes the badge shown in the preview panel for a given availability state and age. The mechanics of Sections 3.9 and 3.10 become something an examiner can manipulate rather than something they must take on trust.

ShopNear

Operator console

Shop approvals

Master catalogue

Order explorer

Disputes

Analytics

Ranking & decay

Ranking & decay controls

Search ranking weights (spec §8) and availability decay thresholds (spec §7). The live re-ranking below runs against real search results — moving a slider visibly reorders them.

Live search

Search term

Location

Radius (m)

atta

Navrangpura (seed centre)

3000

Run search

Ranking weights

w1-availability + w2-proximity + w3-rating + w4-open-now (spec §8). Drag a slider — re-ranking happens instantly, client-side, over the search results above.

Availability confidence

Proximity

Shop rating

Open now

0.40

0.30

0.20

0.10

Reset to server defaults

This re-ranks the results already fetched above, live, in your browser — a faithful copy of the server's own scoring formula. It does not persist: PUT /api/admin/config/ranking-weights does not exist yet, so moving these sliders never changes what the customer app itself returns from the server. Run a fresh search after changing weights to see it applied to new results too.

Decay threshold preview

Shows exactly how a badge would look for a given availability + age, using the same rule table as apps/api/src/modules/availability/confidence.ts (spec §7).

"In stock" → "Likely available" after (hours)

Merchant last said

2 h

In stock

Reset to server defaults

Fig. 5.13 — Ranking weight and decay threshold controls

ShopNear

Operator console

Shop approvals

Master catalogue

Order explorer

Disputes

Analytics

Ranking & decay

Dispute queue

Resolve or reject customer disputes with an admin note. Live against the API.

Open

Resolved

Rejected

All

Open Price mismatch Order SN-2427

Shop did not have the item when I reached, despite the confirmation.

Raised 05 Sept 2026, 07:36 am · order total ₹1,013.91

Admin note (required to resolve or reject)

e.g. Refunded via cash on next visit; shop confirmed stock error.

Resolve Reject

Open Price mismatch Order SN-2478

Shop did not have the item when I reached, despite the confirmation.

Raised 04 Sept 2026, 05:46 pm · order total ₹88.3

Admin note (required to resolve or reject)

e.g. Refunded via cash on next visit; shop confirmed stock error.

Resolve Reject

ShopNear Admin

admin@shopnear.local

Log out

Fig. 5.14 — Dispute resolution queue

5.7 Realtime Layer and Scheduled Jobs

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING, PIT VADODARA

51

Socket.IO is initialised on the same HTTP server as the REST API. The handshake carries the same access token as a REST request and is verified the same way; shop ownership is checked rather than inferred from the role claim. Clients are then joined to per-shop and per-order rooms, so that a merchant could receive events only for shops they own and a customer only for their own orders — the room topology is itself an authorisation boundary rather than a broadcast convenience. Two events are emitted: `order:new` to the shop room when a reservation is placed, and `order:updated` to the order room on every status change.

Implementation status — how live updates actually reach the screen today

The realtime channel above is implemented on the API and covered by integration tests, but **none of the three clients currently subscribes to it**; `socket.io-client` is a development dependency of the API only. The live-updating behaviour the user sees is produced instead by short-interval revalidation in TanStack Query: the customer's order-tracking screen refetches every 3 seconds and stops once the order reaches a terminal state, and the merchant's incoming-order badge refetches every 5 seconds. The observable result — a status change appearing without any refresh — is genuine and was verified end to end, but the mechanism is polling, not push. Connecting the clients to the existing channel is the natural next step and is recorded in Section 7.3.

Two scheduled jobs run under `node-cron`. The **decay job** runs hourly, moving availability records whose timestamps have aged past the configured thresholds towards `USUALLY_AVAILABLE` and writing an `AUTO_DECAY` event for each. The **expiry job** sweeps reservations that have passed their expiry instant without a merchant response and moves them to `EXPIRED` through the same state machine that governs every other transition. Both jobs read the current instant from the clock service rather than the system clock, which is what allows time travel to exercise them on demand.

5.8 Security Implementation

Security measures are proportionate to a demonstration system, but each is real rather than nominal.

- **Password storage.** Argon2 hashing for merchant and administrator credentials. Customers hold no password at all, authenticating by one-time password, which removes an entire category of risk.
- **Token model.** Short-lived access tokens with longer-lived refresh tokens, signed with separate secrets held in environment configuration and never committed to version control.
- **Role enforcement.** Middleware validates the role claim before any handler executes. A customer token presented to a merchant endpoint receives HTTP 403.
- **Cross-tenant isolation.** Shop ownership is verified against the authenticated principal for every shop-scoped and order-scoped operation, so one merchant cannot read or modify another merchant's orders or inventory. This is covered by explicit automated tests rather than assumed.
- **Input validation.** Every endpoint validates its body, parameters and query against a Zod schema before the handler runs; the same schemas type the client, so the two ends cannot drift

apart.

- **Rate limiting.** Authentication routes are rate-limited, since one-time-password and password guessing are the obvious brute-force targets.
- **Transport and headers.** Security headers via helmet and an explicit cross-origin policy.
- **Error hygiene.** A single error envelope; no stack trace or internal identifier is ever returned to a client.

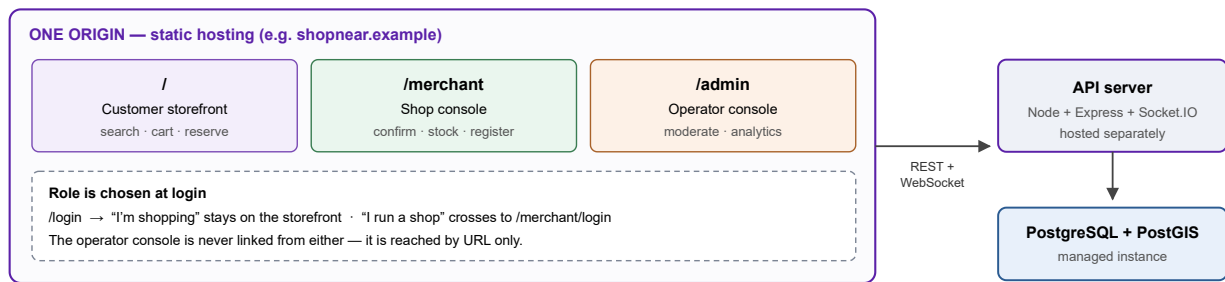
Module	Lines of code	Contents
Customer application	10,395	Screens, design system, state and demonstration catalogue
API server	6,837	Domain modules, middleware, jobs, realtime
Database schema and seed	4,627	Prisma schema, six migrations, deterministic seed generator
Merchant application	4,364	Console, registration wizard, three translation catalogues
Administrator console	2,413	Moderation, analytics, ranking and decay controls
Shared contracts package	644	Zod schemas and types used by the API and all three clients
Total	29,280	TypeScript and TSX, excluding dependencies and generated code

Table 5.3 — Source code size by module

5.9 Deployment Topology

The three clients are separate applications, but presenting them to a user as three separate web addresses would be poor. A customer who is also a shopkeeper should not need to remember two domains, and a demonstration is far easier to give from a single link. The three bundles are therefore **built into one deployable directory and served from one origin**, mounted at different paths.

This keeps the architectural separation described in Section 3.1 — each client still has its own bundle, its own route guard and its own login — while unifying the hosting. Two settings make it work, both applied only at build time so that development still serves each application at its own root: Vite’s `base` option rewrites asset URLs into the subpath, and the router reads the same value as its `basename` so that in-app navigation stays inside the subtree. The host is configured to fall back to the correct `index.html` per subtree, so a deep link such as `/merchant/inventory` resolves rather than returning a 404.



The customer storefront alone can be deployed with no backend at all, because it falls back to a client-side catalogue.
 The merchant and operator consoles have no such fallback — they require the API and database to be reachable.

Fig. 5.15 — Deployment topology: three bundles, one origin

The role choice happens where the user is already stopping — on the login screen — rather than on a separate landing page, so browsing the catalogue costs no additional interaction. Each login screen carries a two-option control and links across to the other; because the two applications are separate bundles, that crossing is a genuine navigation rather than an in-app route change.

One asymmetry is worth recording. The customer storefront can be deployed entirely on its own, because it falls back to a client-side catalogue when no API is configured. The merchant and operator consoles have no equivalent fallback: they read and write live data on every screen, so deploying them requires the API server and the PostGIS database to be hosted and reachable as well. This is why a static-only deployment can show the storefront but not the other two.

CHAPTER VI

TESTING AND VALIDATION

6.1 Testing Strategy

Testing was treated as part of the deliverable rather than as a closing activity. The guiding principle was that any claim made about the system's behaviour should be reproducible by running a command, and that the rules most expensive to get wrong should be the ones tested most thoroughly.

Three levels of testing were applied.

Unit tests over pure domain logic. The confidence derivation, the ranking score, the opening-hours calculation and the state transition table are implemented as pure functions with no database dependency. They can therefore be tested exhaustively and cheaply. The state machine in particular is tested against its *entire* transition space — not only that legal transitions succeed, but that every illegal one is rejected, and that a reserve-and-collect order cannot reach a delivery-only state.

Integration tests over HTTP. Endpoint behaviour is exercised through Supertest against the real Express application and the real PostgreSQL database, not against mocks. This is the only way to establish that a JWT is genuinely rejected, that a PostGIS radius query genuinely excludes a distant shop, and that a transaction genuinely rolls back. These tests create and clean up their own fixtures.

Data-integrity and schema tests. A separate group asserts that the schema itself behaves as designed: that the composite uniqueness constraint permits one phone number to hold both a customer and a merchant account but not two of either, that cascade deletes propagate correctly, and that the seed generator is deterministic across runs.

6.2 Distribution of the Test Suite

Area under test	Tests	What is verified
Client catalogue fixtures and radius behaviour	152	Catalogue density per shop type, results present at every radius rung, shop-scoped catalogue correctness
Seed determinism and data realism	50	Reproducible pseudo-random output, geographic offsets, catalogue and order-history realism
Order state machine and lifecycle	41	Every legal transition, every illegal transition, per-order-type restrictions, expiry and cancellation

Area under test	Tests	What is verified
Search, ranking and shop discovery	31	Radius filtering, keyword and trigram matching, composite ordering, multi-item coverage, computed opening status
Schema and database integrity	27	Composite uniqueness, cascade behaviour, referential constraints, PostGIS availability
Authentication and authorisation	25	One-time-password flow, password login, token refresh, role rejection with HTTP 403, cross-tenant isolation
Infrastructure (clock, runtime config, errors, validation, realtime)	18	Clock offset persistence, settings override and fallback, error envelope shape, schema rejection, socket delivery
Availability confidence and decay	15	Every row of the badge rule table, hourly decay behaviour, reservation expiry sweep
Reviews and disputes	10	One review per order, rating recalculation, dispute raising and resolution
Merchant registration	6	Atomic multi-record creation, pending status on submission, starter catalogue selection
Shared contract package	4	Enumerations agree between client and server
Total	379	35 test files, all passing

Table 6.1 — Distribution of automated tests

6.3 Representative Test Cases

ID	Test case	Expected result	Actual result	Status
T01	Customer submits the correct one-time password	Access and refresh tokens issued; role claim is CUSTOMER	As expected	Pass
T02	Customer token sent to a merchant-only endpoint	Request rejected with HTTP 403	As expected	Pass
T03	Merchant A requests merchant B's shop orders	Request rejected; no data disclosed	As expected	Pass
T04	Same phone number registers as customer and as merchant	Both permitted; a second account in the same role is rejected	As expected	Pass

ID	Test case	Expected result	Actual result	Status
T05	Search for “ata” (misspelling)	Trigram similarity returns wheat-flour products	As expected	Pass
T06	Search with a 250 m radius against a shop 2 km away	Distant shop excluded from results	As expected	Pass
T07	IN_STOCK record aged 30 minutes	Badge reads “In stock” with the confirmation age stated	As expected	Pass
T08	IN_STOCK record aged 30 hours	Badge degrades to “Usually available”	As expected	Pass
T09	OUT_OF_STOCK record aged 20 hours	Badge becomes “Usually available” — restock assumed	As expected	Pass
T10	No availability record exists for a product	Badge reads “Ask the shop”; reservation still permitted	As expected	Pass
T11	Transition PLACED → COMPLETED attempted directly	Rejected with HTTP 400 before any write	As expected	Pass
T12	READY_FOR_PICKUP attempted on a delivery order	Rejected — state is valid only for reserve-and-collect	As expected	Pass
T13	Transition out of a terminal state attempted	Rejected for all four terminal states	As expected	Pass
T14	Reservation left unanswered past its expiry instant	Expiry job moves the order to EXPIRED	As expected	Pass
T15	Merchant marks a line “Don’t have it”	Line resolved, inventory state updated, audit event appended	As expected	Pass
T16	Decay job run after the clock is advanced 30 hours	Stale records decay; AUTO_DECAY events written	As expected	Pass
T17	Shopping list of five items searched	Shop with greatest coverage identified; remainder split across a second shop	As expected	Pass
T18	Merchant registration wizard submitted	User, shop, address and inventory created atomically; status PENDING	As expected	Pass

ID	Test case	Expected result	Actual result	Status
T19	Second review submitted for the same order	Rejected; one review per order enforced by unique constraint	As expected	Pass
T20	Database reset executed twice consecutively	Identical content hash over the inventory join	As expected	Pass

Table 6.2 — Representative test cases and outcomes

6.4 Test Execution Results

The complete suite is executed with a single command, `npm test`, which runs Vitest across every workspace. All 379 tests across 35 files pass, in approximately 49 seconds on the development machine.

```

✓ apps/api/src/modules/orders/stateMachine.test.ts (26 tests) 8ms
✓ apps/api/src/modules/auth/auth.test.ts (18 tests) 1614ms
✓ apps/api/src/modules/orders/orders.test.ts (15 tests) 2260ms
✓ apps/api/src/modules/availability/confidence.test.ts (10 tests) 4ms
✓ apps/api/src/modules/shops/shops.test.ts (9 tests) 1103ms
✓ apps/api/src/http/authMiddleware.test.ts (7 tests) 216ms
✓ apps/api/src/modules/search/search.test.ts (6 tests) 900ms
✓ apps/api/src/modules/search/multiSearch.test.ts (3 tests) 483ms
✓ apps/api/src/jobs/decay.job.test.ts (3 tests) 589ms
... 26 further files

Test Files  35 passed (35)
  Tests    379 passed (379)
Duration   48.73s

```

Fig. 6.1 — Test suite execution output

Manual validation. Alongside the automated suite, the following were confirmed by hand: the health endpoint reports the live PostGIS version; a search returns multiple shops carrying the same product at differing prices and differing confidence badges; confirming an order in the merchant console updates the customer’s screen without a refresh (by short-interval revalidation — see Section 5.7); advancing the clock visibly degrades confidence badges; the merchant console renders correctly in all three languages; and every screenshot in Chapter V was captured from these running applications.

6.5 End-to-End Journey Verification

Automated unit and integration tests establish that each part behaves correctly in isolation. To confirm that the parts compose, a single reservation was driven through the running system from the

customer interface to the merchant interface and back, with the database inspected afterwards. Table 6.3 records that run.

#	Action	Observed result
1	Customer signs in with phone and one-time password	Tokens issued; redirected to the account screen
2	Opens a shop 80 m away (Shreeji Kirana)	Shop catalogue rendered with prices and confidence badges
3	Adds an item to the cart	Aashirvaad Multigrain Atta 5 kg, ₹313.64, badge “Usually available”
4	Chooses reserve-and-collect with cash on pickup, submits	Order SN-2521 created; pickup code 6324 ; two-hour expiry counting down
5	Merchant signs in on the shop console	Authenticated as the owner of Shreeji Kirana
6	Opens the incoming-reservations screen	SN-2521 present and awaiting a response
7	Marks the line “Have it” and confirms the order	Order accepted; merchant screen clears
8	Customer screen, never reloaded	Progress tracker advanced to <i>Placed</i> → <i>Confirmed</i> → <i>Ready for pickup</i>
9	Database inspected	<code>status = READY_FOR_PICKUP</code> , <code>item fulfilmentStatus = AVAILABLE</code> , <code>confirmedAt</code> and <code>readyAt</code> both set
10	Audit log inspected	New <code>AvailabilityEvent</code> : <code>IN_STOCK</code> → <code>IN_STOCK</code> , source <code>RESERVATION_CONFIRMED</code>

Table 6.3 — Recorded end-to-end reservation journey

Two results in this table matter more than the rest. Step 8 confirms that a merchant action reaches the customer’s open screen with no interaction on their part. Step 10 confirms the mechanism at the heart of the project: the merchant’s single tap did not merely resolve one order line, it refreshed that shop’s availability record for that product and appended an entry to the audit log — which is what every subsequent customer’s confidence badge is computed from. Step 9 also demonstrates the automatic advance from `CONFIRMED` to `READY_FOR_PICKUP` for reserve-and-collect orders, described in Section 3.12.

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

ShopNear set out to answer a narrow question well: which neighbourhood shop near me has this item right now, and how far should I trust that answer. The resulting system is a TypeScript monorepo in which one Express and PostgreSQL/PostGIS API serves three independent React clients, comprising roughly 29,000 lines of code and validated by 379 automated tests, all passing.

The contribution that distinguishes the project is not any individual feature but a modelling decision: **availability is represented as a confidence level with a timestamp, never as a stock count**. This single choice propagates through the entire system. It is why the merchant interaction can be reduced to one tap, why a scheduled job must decay stale signals, why the customer interface displays “confirmed 20 minutes ago” rather than a bare label, why the ranking function accepts confidence as its heaviest input, and why an append-only event log exists at all. A project that had instead modelled a quantity would have been simpler to build and would have produced data that was confidently wrong.

The three algorithmic components — the confidence derivation, the composite ranking function and the greedy multi-item coverage search — are implemented as pure functions, tested exhaustively, and exposed through the administrator console in a form that can be manipulated live rather than merely asserted. The order state machine ensures that the one part of the system where mistakes cost real time or money cannot enter an inconsistent state.

7.2 Key Findings

- **Modelling uncertainty explicitly is more useful than hiding it.** Displaying the age of an availability signal and letting it decay produces a system that remains trustworthy even when its data is old, because the customer is told the data is old. A system that presents stale data as fresh forfeits trust permanently at the first wrong answer.
- **Merchant effort is the binding constraint, not merchant willingness.** Designing every merchant interaction around a single tap, and translating the console into Hindi and Gujarati, addresses the actual obstacle to adoption. No feature that adds a second step to the confirmation flow survived design review.
- **The global-catalogue and per-shop-inventory split is load-bearing.** Because a product exists once globally and is offered many times locally, cross-shop price and availability comparison is a straightforward join. Had each shop held its own product records, the headline comparison feature would have been impossible.
- **Doing geospatial work in the database rather than in application code is decisively correct.** A GiST-indexed geography column with `ST_DWithin` gives exact geodesic distance in an index-

accelerated query; the application-side haversine alternative cannot use an index at all.

- **A deterministic seed is a genuine engineering asset.** Identical demonstration data on every machine and every run makes defects reproducible, tests stable and presentations predictable.
- **Multi-item search is only possible because the platform indexes many shops.** Minimising the number of stops required by a shopping list is a problem a single-warehouse system does not have and therefore cannot solve.

7.3 Limitations

The following limitations are stated plainly, because a system whose boundaries are known is more defensible than one whose claims exceed what it does.

- **Demonstration data only.** The platform operates on a generated dataset of 14 shops in one Ahmedabad neighbourhood. No real shop, merchant, customer or payment is involved anywhere in the system.
- **The customer application runs against a demonstration catalogue by default.** It ships configured to use a client-side fixture catalogue so that it can be shown with no database and no API process running. A single environment flag switches it to the live backend, and both paths implement the identical interface — but the default is the fixture catalogue, and this should not be misread as the API serving those screens.
- **Merchant verification is not performed.** Uploaded licence and registration documents are checked for file format only and stored locally. Nothing is verified against any authority. The interface labels this explicitly rather than implying a check that does not occur.
- **Payment is not integrated.** Payment mode is recorded and a mock online option exists, but no payment gateway is connected and no money moves. The intended real-world flow is cash paid in person at the shop.
- **Some administrative endpoints are not yet built.** The shop approval queue and several analytics panels are fully implemented on the client against a defined contract, but the corresponding API routes do not exist yet. Rather than display placeholder figures, these panels state the gap and name the missing endpoint (Figure 7.1). This is a deliberate choice: an interface that invents numbers is worse than one that admits a gap.
- **Live updates are polled, not pushed.** The API carries a complete, authenticated Socket.IO channel with room-scoped order events, and it is covered by tests, but no client subscribes to it yet. Screens stay current through 3- and 5-second revalidation instead. This is invisible to the user at demonstration scale and would become a real cost — in wasted requests and in latency — only with many idle open sessions.
- **Availability depends entirely on merchant participation.** A shopkeeper who stops answering causes their confidence badges to decay to “usually available”. The system degrades honestly rather than misleadingly, but it cannot manufacture freshness that no one supplied.

- **The demonstration control panel is not yet built.** The clock service that would drive time travel is complete and used throughout, but the operator panel wrapping it, together with simulated traffic and scripted scenarios, remains outstanding.
- **Scale is untested.** Performance has been measured only against the seeded dataset. The indexing strategy is designed for larger volumes, but no load testing has been carried out.

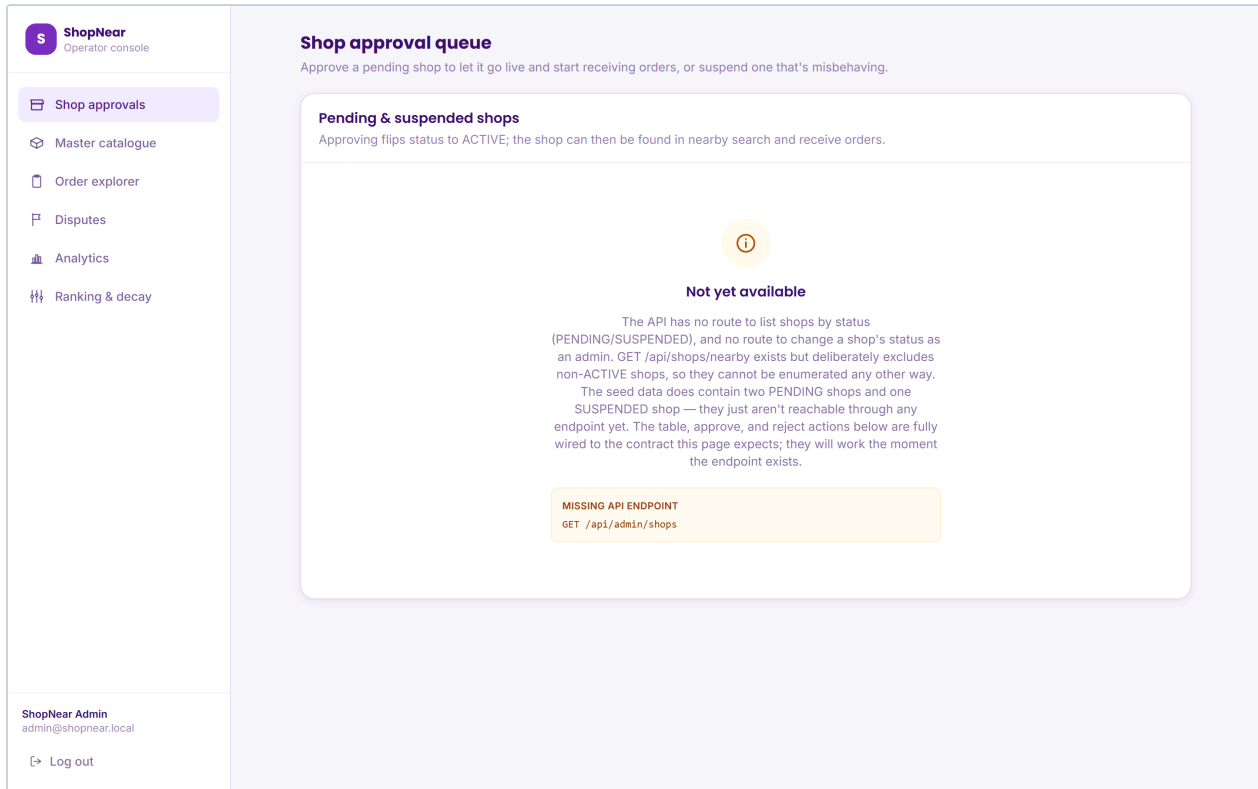


Fig. 7.1 — Where an endpoint does not yet exist, the console names it rather than displaying invented data

7.4 Future Scope

Completing the administrative surface. The immediate next work is the remaining administrative endpoints — shop listing and status change, aggregated order analytics, catalogue duplicate detection and merge — so that the client screens already built against those contracts become live.

Subscribing the clients to the realtime channel. The server-side push infrastructure already exists; the remaining work is a client-side subscription that invalidates the relevant query on `order:new` and `order:updated`, replacing the polling described in Section 5.7 and removing its request overhead.

Integrated payments. A UPI integration would let the customer pay when collecting without handling cash, while preserving the in-person collection model. Reservation deposits could reduce the no-show rate that any reservation system faces.

Point-of-sale integration. Where a shop already runs billing software or a barcode scanner, availability could be inferred from actual sales rather than requiring a tap. This would not replace the confidence model but would feed it a second, higher-frequency signal source — and the source

field already present on every availability event exists precisely so that such a signal can be distinguished from a manual one.

Predictive confidence. The append-only availability event log accumulates a history of how each shop's stock behaves over time. With sufficient history, confidence could be predicted from a shop's observed restocking pattern for a product rather than derived from a fixed threshold table, so that a shop which reliably restocks flour every Tuesday would be scored accordingly.

ONDC interoperability. Publishing shop catalogues to the Open Network for Digital Commerce would let ShopNear shops be discovered by any buyer application on that network. The availability-confidence model is a natural contribution to such a network, since the problem of small sellers producing unreliable stock data is not unique to this platform.

Delivery coordination. The data model already supports delivery as an order type. A future version could coordinate a shared neighbourhood delivery person across several shops, rather than requiring each shop to arrange delivery independently.

Broader language and accessibility coverage. Extending full translation to the customer application, adding further regional languages, and adding voice input for shopping lists would materially widen the addressable user base.

Progressive web application packaging. Installability and offline caching of the last search result would suit a use case where the customer is frequently outdoors on an unreliable connection.

CHAPTER VIII

REFERENCES

1. Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6), 80–83.
2. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. Internet Engineering Task Force, RFC 7519. <https://datatracker.ietf.org/doc/html/rfc7519>
3. Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. *IEEE European Symposium on Security and Privacy*, 292–302.
4. PostGIS Project Steering Committee. (2026). *PostGIS 3.4 Manual — Geography Type and Spatial Indexing*. <https://postgis.net/documentation/>
5. PostgreSQL Global Development Group. (2026). *PostgreSQL 16 Documentation — pg_trgm, GiST and GIN Index Types*. <https://www.postgresql.org/docs/16/>
6. Guttman, A. (1984). R-Trees: A Dynamic Index Structure for Spatial Searching. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 47–57.
7. Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). An Analysis of Approximations for Maximizing Submodular Set Functions. *Mathematical Programming*, 14(1), 265–294. (Basis for the greedy coverage approximation used in multi-item search.)
8. Ukkonen, E. (1992). Approximate String Matching with q-grams and Maximal Matches. *Theoretical Computer Science*, 92(1), 191–211.
9. Vincenty, T. (1975). Direct and Inverse Solutions of Geodesics on the Ellipsoid with Application of Nested Equations. *Survey Review*, 23(176), 88–93.
10. Ray, A., & Singh, P. (2022). A Study on the Growth of Hyper-Local E-Commerce and Quick Commerce in Modern Retail. *Journal of Business and Retail Management Research*.
11. Open Network for Digital Commerce. (2026). *ONDC Protocol Specifications and Network Policy*. <https://ondc.org>
12. Meta Open Source. (2026). *React Documentation*. <https://react.dev>
13. Prisma Data, Inc. (2026). *Prisma ORM Documentation — Schema, Migrations and Client*. <https://www.prisma.io/docs>
14. OpenJS Foundation. (2026). *Express: Fast, Unopinionated, Minimalist Web Framework for Node.js*. <https://expressjs.com>
15. Socket.IO. (2026). *Socket.IO Documentation — Rooms, Namespaces and Transports*. <https://socket.io/docs/v4/>
16. Vite. (2026). *Vite Documentation — Next Generation Frontend Tooling*. <https://vitejs.dev>

17. Tailwind Labs. (2026). *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>
18. Vitest. (2026). *Vitest Documentation — A Vite-Native Testing Framework*. <https://vitest.dev>
19. World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/WCAG22/>
20. Mozilla Developer Network. (2026). *Using the Fetch API and RESTful Web Services*. <https://developer.mozilla.org>